

Lab 5 : Moving and Animating a Character

Lab Overview:

In this lab you will bring a character to life in two stages. First, you will write a physics-based **PlayerMovement** script from scratch so the player can walk, jump and collide with the world using a Rigidbody. Then you will use Unity's Animator system to configure an animation state machine, build a blend tree, create transitions between animations, and write a second script that drives those animations from the character's real movement. Finally, you will create your own animation clip for a spinning coin that adds to the player's score when collected.

By the end of the lab you should be able to explain why input is read in `Update()` but physics is applied in `FixedUpdate()`, how a raycast ground check works, and how gameplay code passes information to an Animator through parameters.

Step-by-Step Solution:

Task 1: Setup the Animator and Avatar for Both the Character and Animations

1. Open the github classroom assignment and download the repo :

https://github.com/amcnabbbaltar/420_541_Lab5_2026

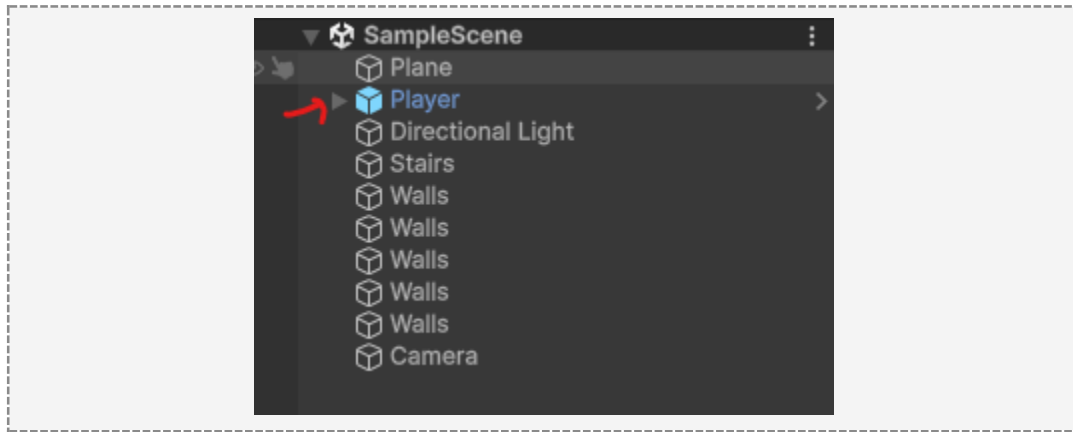
2. Once the repo is clone , Open the Sample Scene

- In the scene folder, Open the sample scene . You should see something like this

Name	Date modified	Type	Size
 SampleScene 	2026-09-23 11:36 PM	Unity scene file	30 KB
 SampleScene.unity.meta	2026-09-23 11:36 PM	META File	1 KB

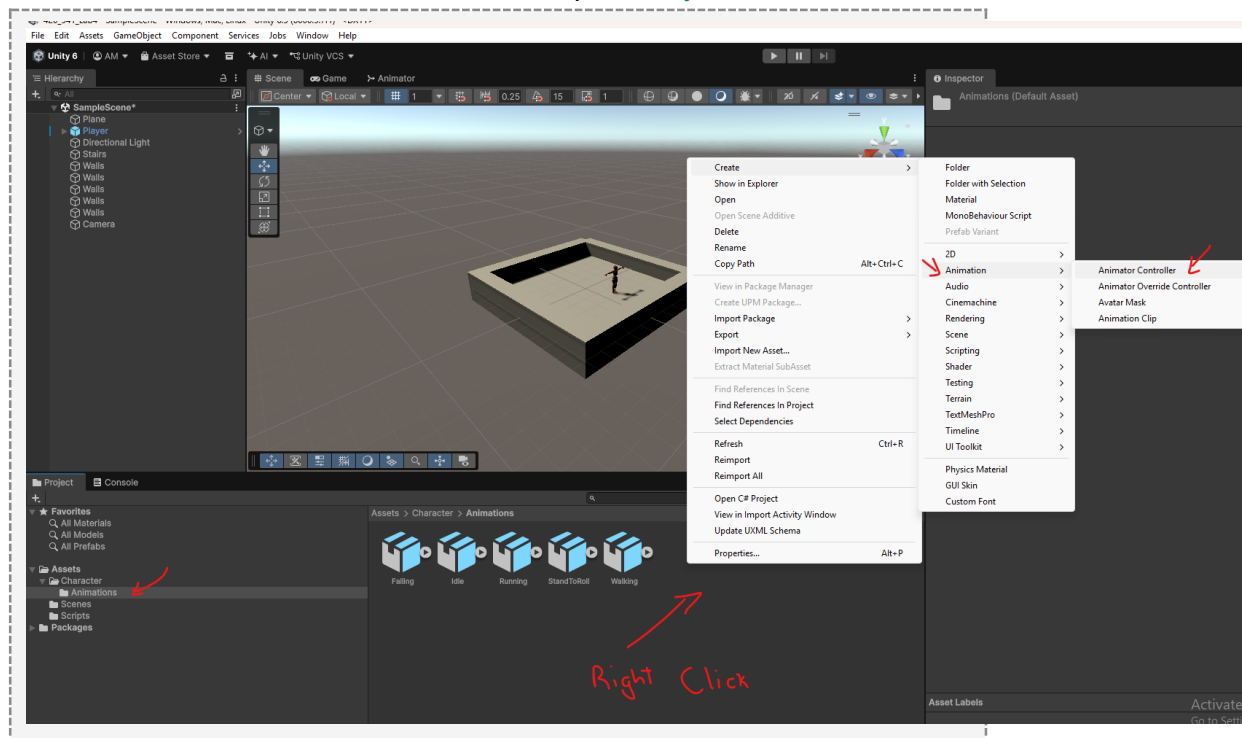
2. **Select Your Character:**

- In Unity, go to the **Hierarchy** window and locate your Player prefab.



3. Create an Animator Controller:

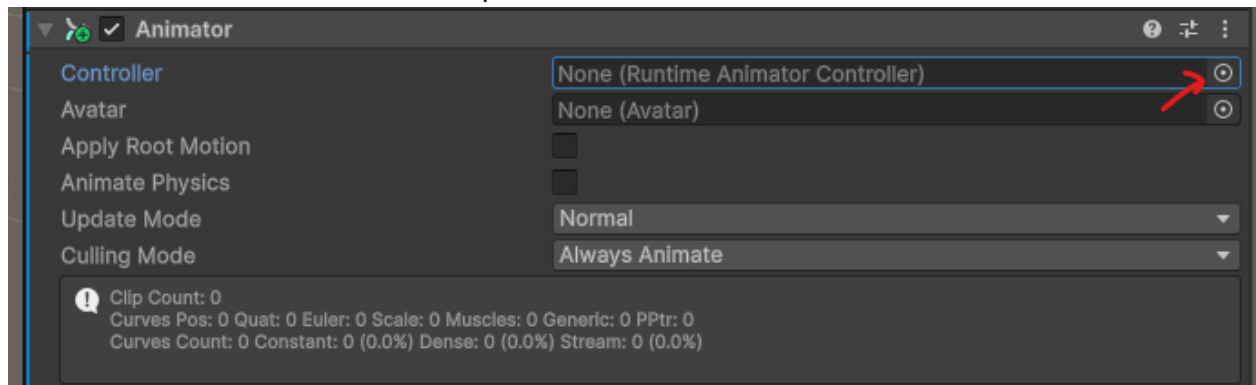
- In the **Assets** window (Project tab), open the **Character** Folder, then the **Animation** Folder
- Then right-click and choose **Create > Animator Controller**.
- Name the new Animator Controller, for example, **PlayerAnimator**.



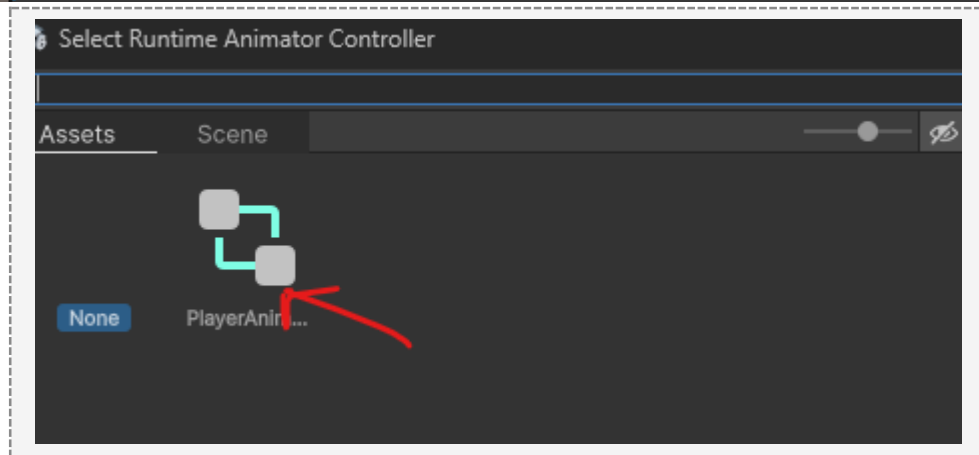


4. Assign the Animator Controller to the Character:

- Select your character in the **Hierarchy** and look in the **Inspector** window.
- Under the **Animator** component, assign the newly created **CharacterAnimator** controller by dragging it from the **Assets** window to the **Controller** slot in the **Animator** component.



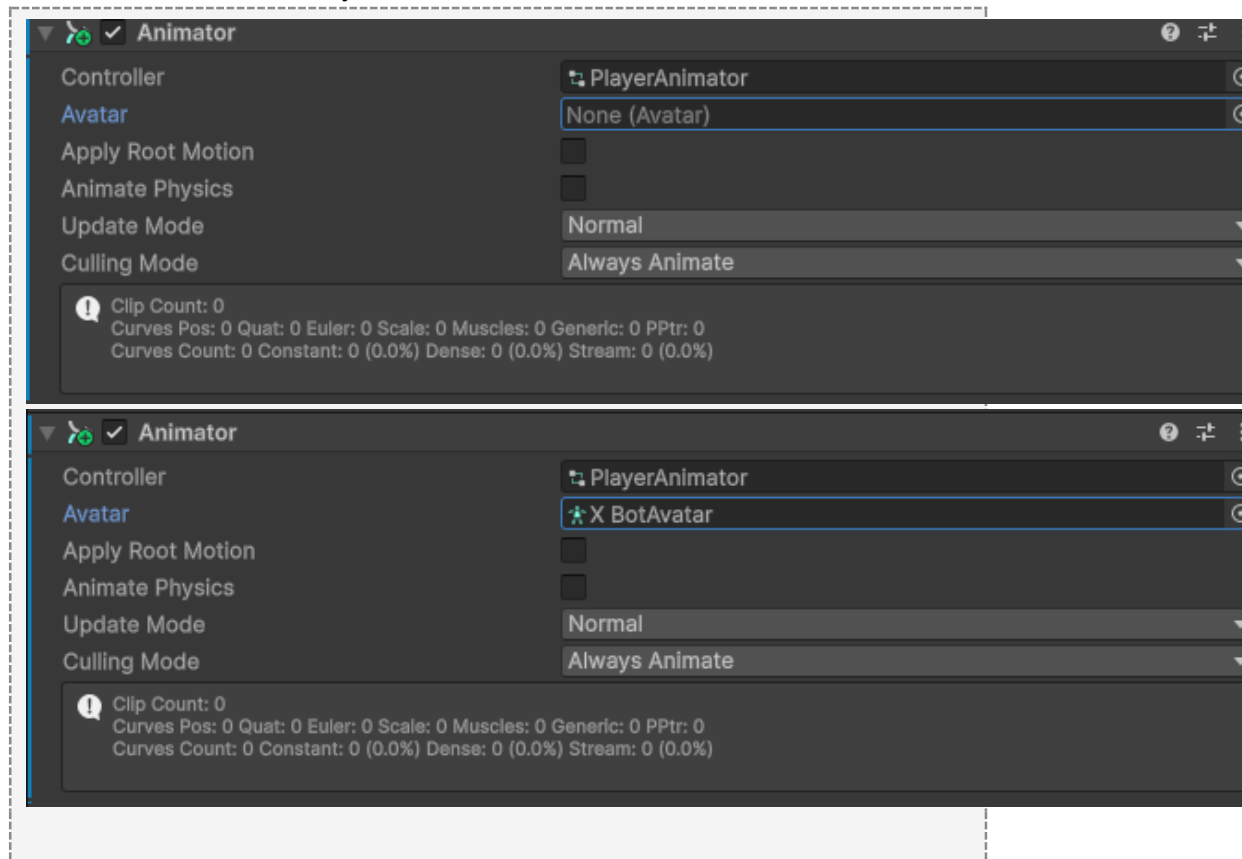
○



5. Ensure the Avatar is Assigned:

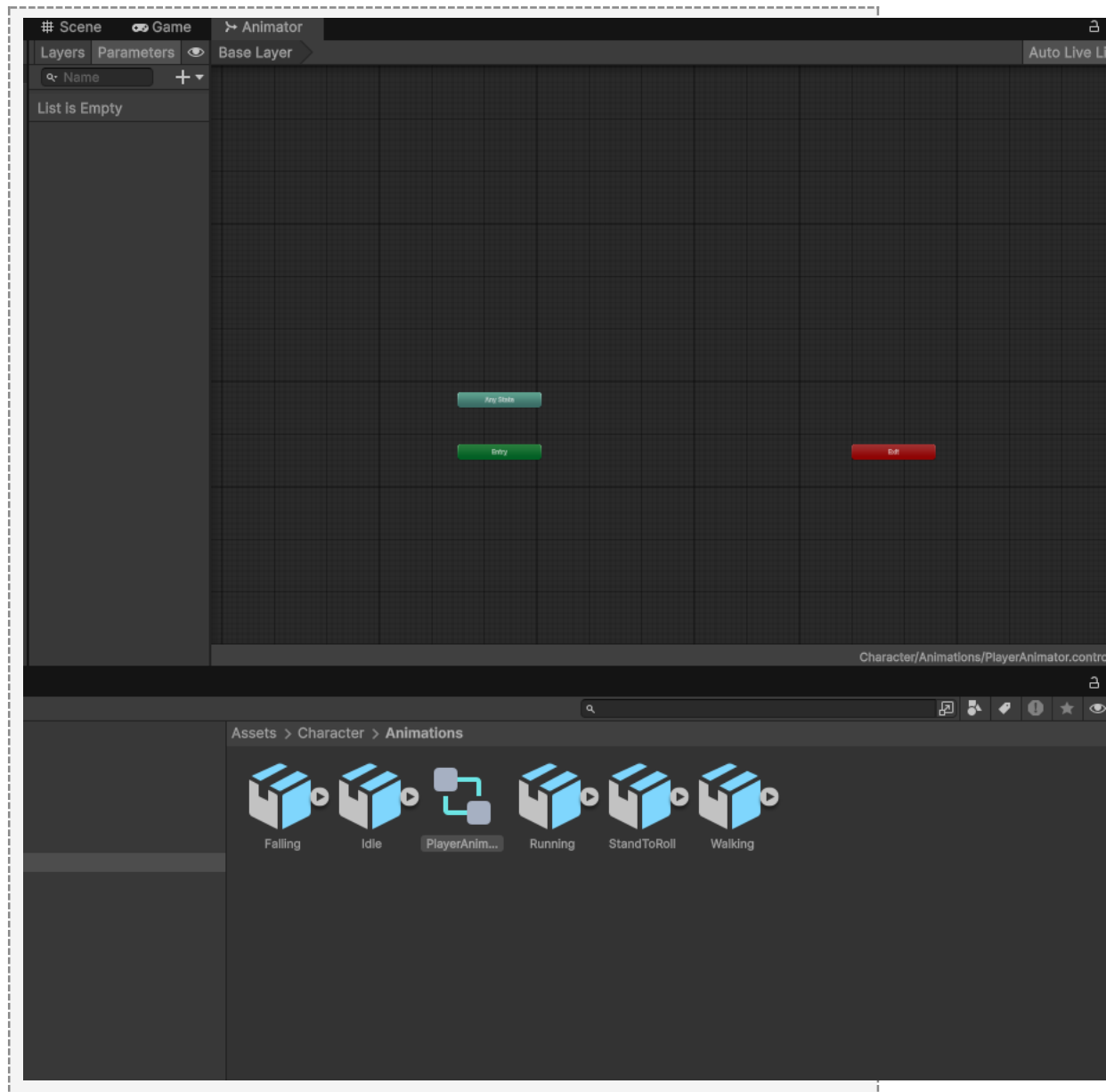
- In the **Animator** component, there is a field for **Avatar**. Ensure that the Avatar (skeleton) for your character is properly assigned here. Unity should auto-assign

it when importing the character, but if it is missing, you can manually select the Avatar file associated with your character model.



6. **Open the Animator Controller:**

- You should already have animations (Idle, Walking, Running, Rolling, Falling) in your **Animation** folder(Assets/Character/Animation).
- Open the **Animator** window by going to **Window > Animation > Animator** or by **double clicking on the CharacterAnimator** in the assets view .
 - i. You should see a window like this that pop up

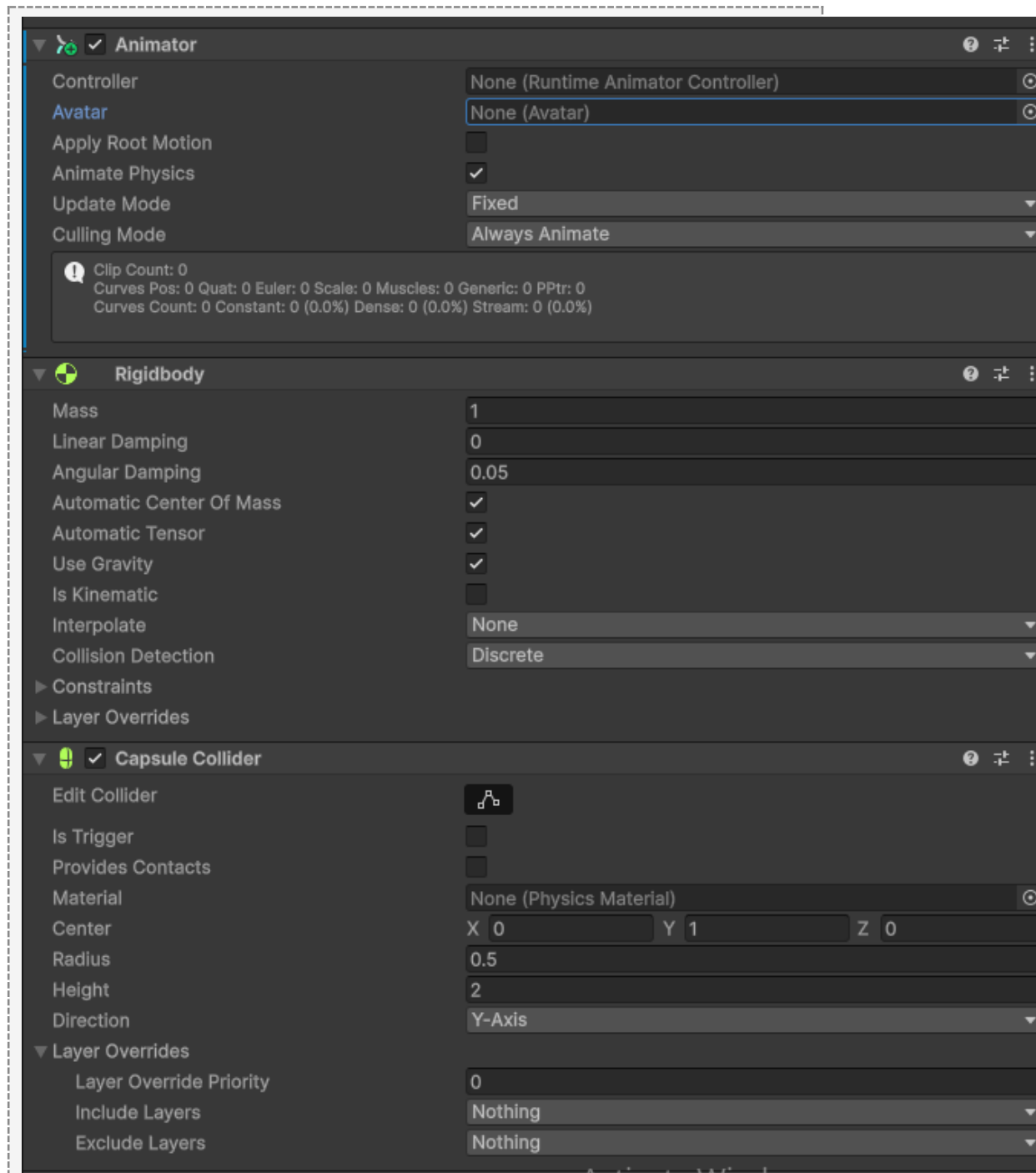


Task 2: Build the Player Movement Script

Before we animate anything, the character needs to actually move. In this task you will write a **PlayerMovement** script that uses Unity's physics engine (a Rigidbody) to move the player with WASD / arrow keys and jump with Space. You will build it one piece at a time so you understand what every line does.

1. Prepare the Player for physics:

- Select your **Player** in the **Hierarchy**.
- If the Player already has an older movement script attached (for example **CharacterMovement**), remove that component (right-click the component header > **Remove Component**) so two scripts don't fight over the same Rigidbody.
- Click **Add Component** and add a **Capsule Collider**. Adjust its **Center** and **Height** until the green capsule wraps the character in the Scene view. For a typical humanoid, try Center Y = 0.9, Height = 1.8, Radius = 0.3.
- In the **Animator** component, **uncheck Apply Root Motion**. Our script, not the animation clips, will decide where the character goes. (In Task 4 the roll behaviour turns root motion on only while rolling.)

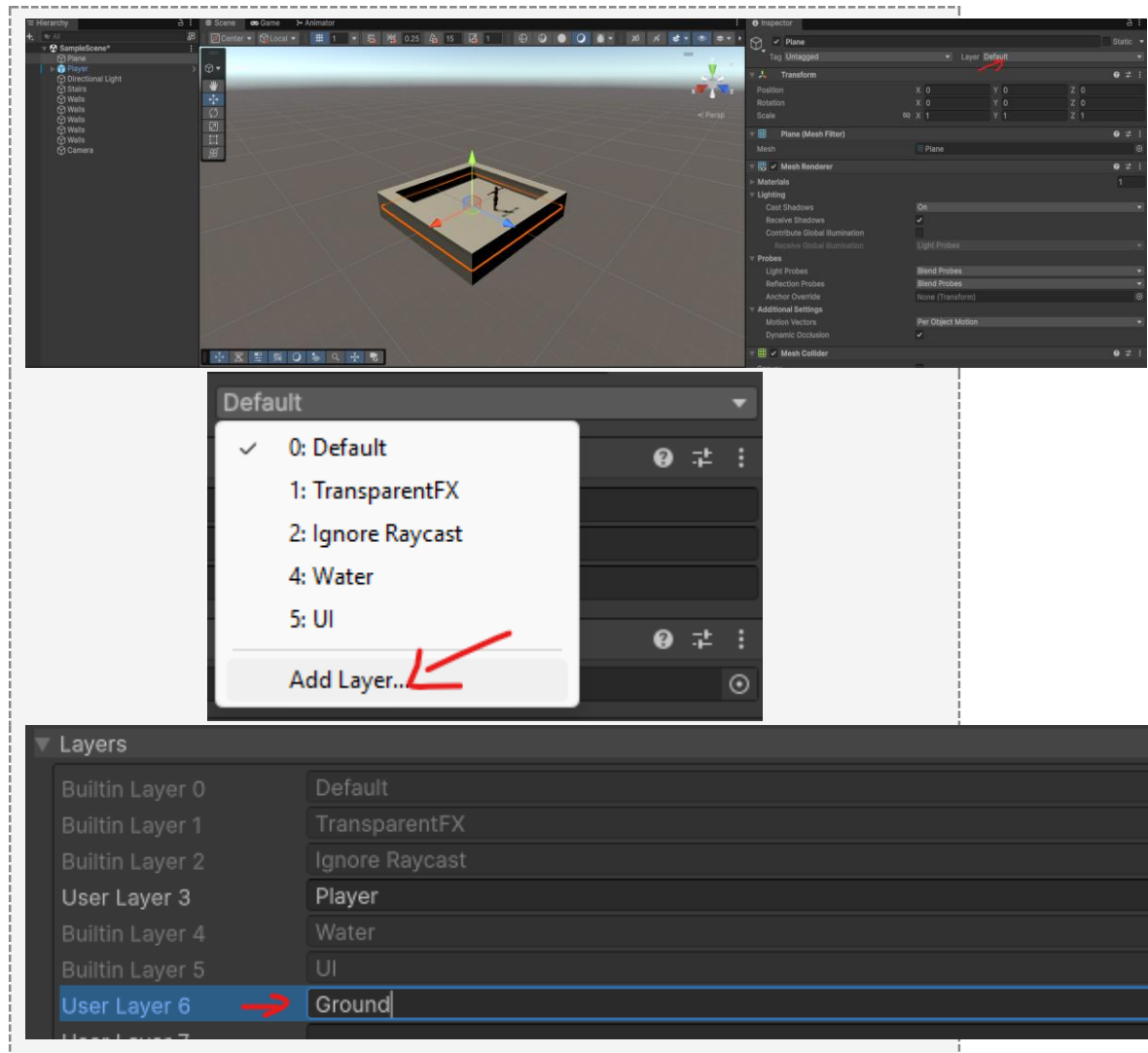


Why a Rigidbody? A Rigidbody hands the character over to Unity's physics engine, which gives us gravity, collisions and forces for free. You don't need to add it yourself: the `[RequireComponent]` line in our script adds it automatically.

2. Create a Ground layer:

- Select the floor/ground object(s) in your scene.

- At the top of the Inspector, open the **Layer** dropdown and choose **Add Layer....**. Type **Ground** into an empty User Layer slot.
- Select the floor again and set its **Layer** to **Ground**. Make sure the floor has a collider (a Box Collider or Mesh Collider).
- Do **not** put the Player on the Ground layer, otherwise the player could detect itself as ground.



3. Create the script:

- With the Player selected, click **Add Component**, type **PlayerMovement**, choose **New Script**, then **Create and Add**.
- Double-click the script to open it in your code editor and delete the default contents. The file name and class name must both be exactly **PlayerMovement**, or Unity won't be able to attach it.

4. Declare the class and its variables:

Start with the class, its settings and the private variables it will use:

```
using UnityEngine;

// Moves the player with a Rigidbody:
// A/D or Left/Right arrows to ROTATE, W/S or Up/Down to move
// Forward/Backward, Space to jump.
[RequireComponent(typeof(Rigidbody))]
public class PlayerMovement : MonoBehaviour
{
    [Header("Movement Settings")]
    [SerializeField] private float moveSpeed = 7f;           // units per second
    [SerializeField] private float turnSpeed = 150f;         // degrees per
second
    [SerializeField] private float jumpForce = 5f;           // strength of the
jump

    [Header("Ground Check Settings")]
    [SerializeField] private float groundDistance = 0.5f;    // ray length
    [SerializeField] private LayerMask groundMask;           // ground layers

    private Rigidbody rb;                                     // the physics body we move
    private Vector3 moveDirection;                           // worked out from input in Update()
    private float turnInput;                                  // horizontal input for rotation
    private bool isGrounded;                                  // true when standing on the ground
    private bool jumpRequested = false;                       // set in Update(), used later
    // The methods from the next steps go here, inside the class
}
```

- `[RequireComponent(typeof(Rigidbody))]` guarantees the GameObject has a Rigidbody, adding one if it is missing.
- `[SerializeField] private` keeps the variable private to this script but still shows it in the Inspector so you can tune it without editing code. `[Header]` just groups the fields under a label.
- `groundMask` is a **LayerMask**: it tells the ground check which layers count as ground.
- `moveDirection`, `isGrounded` and `jumpRequested` store information that `Update()` collects and `FixedUpdate()` uses.

5. Get the Rigidbody in Start():

Replace the comment inside the class with the `Start()` method:

```
// Start runs once, just before the first frame
private void Start()
{
    // Find the Rigidbody on this GameObject so we can move it
    rb = GetComponent<Rigidbody>();

    // Freeze rotation so the player doesn't fall over when colliding
    rb.constraints = RigidbodyConstraints.FreezeRotation;
}
```

- `GetComponent<Rigidbody>()` finds the Rigidbody on this GameObject and stores it in `rb` so we don't search for it every frame.
- Freezing rotation stops physics from tipping the capsule over when it bumps into walls.

6. Read input and check the ground in Update():

Add the `Update()` method below `Start()`:

```
private void Update()
{
    // 1. Ground Check

    isGrounded = Physics.Raycast(transform.position +
transform.up*groundDistance/2, -transform.up, groundDistance, groundMask);

    // 2. Read Inputs

    turnInput = Input.GetAxisRaw("Horizontal"); // A/D or Left/Right
turn input

    float moveZ = Input.GetAxisRaw("Vertical"); // W/S or Up/Down
forward/backward

    // Rotate the player transform in Update for responsive turning
visuals

    transform.Rotate(0f, turnInput * turnSpeed * Time.deltaTime, 0f);

    // Calculate forward movement relative to current facing direction
    moveDirection = transform.forward * moveZ;
```

```
// 3. Handle Jump Input

if (Input.GetButtonDown("Jump") && isGrounded)
{
    jumpRequested = true;
}
}
```

- **Ground check:** `Physics.Raycast` fires an invisible ray from the character's position straight down (`-Vector3.up`) for `groundDistance` units, and only hits objects on the layers in `groundMask`. If it hits, `isGrounded` is true.
- **Movement input:** `GetAxisRaw` returns -1, 0 or 1 for the Horizontal (A/D, ←/→) and Vertical (W/S, ↑/↓) axes. Multiplying by `transform.right` and `transform.forward` makes movement relative to the way the player is facing.
- `.normalized` gives the direction a length of 1, so moving diagonally isn't faster than moving straight.
- **Jump input:** `GetButtonDown` is only true on the single frame the key is pressed. We only accept it when `isGrounded` is true, so the player can't jump again in mid-air, and we set a flag instead of jumping right away.

7. Apply physics in `FixedUpdate()`:

Add the `FixedUpdate()` method below `Update()`:

```
private void FixedUpdate()
{
    MovePlayer();

    if (jumpRequested)
    {
        Jump();
        jumpRequested = false;
    }
}
```

Why two update methods? `Update()` runs once per rendered frame, which is the only reliable place to catch a key press. `FixedUpdate()` runs on the physics engine's fixed timestep, which is the correct place to change a Rigidbody. The `jumpRequested` flag carries the key press from one to the other, so no jump is ever missed and physics stays smooth.

8. Write the `MovePlayer()` and `Jump()` helper methods:

Finally, add these two methods below `FixedUpdate()`:

```
private void MovePlayer()
{
    // Calculate velocity based on current forward vector
    Vector3 targetVelocity = moveDirection * moveSpeed;

    // Apply movement while preserving vertical velocity
    (gravity/jumping)
    rb.linearVelocity = new Vector3(targetVelocity.x,
    rb.linearVelocity.y, targetVelocity.z);
}

private void Jump()
{
    rb.linearVelocity = new Vector3(rb.linearVelocity.x, 0f,
    rb.linearVelocity.z);

    rb.AddForce(Vector3.up * jumpForce, ForceMode.VelocityChange);
}
```

- **MovePlayer()** sets the horizontal (X and Z) velocity directly for snappy, responsive movement, but keeps the current Y velocity so gravity and jumping still work.
- **Jump()** first zeroes the vertical velocity so every jump reaches the same height, even if the player was falling, then pushes the Rigidbody upward. `ForceMode.VelocityChange` applies the push instantly and ignores the Rigidbody's mass.

Unity 6 note: This lab is written for Unity 6, where the Rigidbody's velocity is called **linearVelocity**. If you are using Unity 2022 or older, write **velocity** instead of **linearVelocity** in both scripts.

9. Checkpoint: compare with the complete script

Save the file. Your finished **PlayerMovement** script should look exactly like this:

```
using UnityEngine;

// Moves the player with a Rigidbody:
// A/D or Left/Right arrows to ROTATE, W/S or Up/Down to move
// Forward/Backward, Space to jump.
[RequireComponent(typeof(Rigidbody))]
public class PlayerMovement : MonoBehaviour
{
    [Header("Movement Settings")]
    [SerializeField] private float moveSpeed = 7f;           // units per second
    [SerializeField] private float turnSpeed = 150f;         // degrees per
second
    [SerializeField] private float jumpForce = 5f;           // strength of the
jump

    [Header("Ground Check Settings")]
    [SerializeField] private float groundDistance = 0.5f;    // ray length
    [SerializeField] private LayerMask groundMask;           // ground layers

    private Rigidbody rb;                                     // the physics body we move
    private Vector3 moveDirection;                           // worked out from input in Update()
    private float turnInput;                                  // horizontal input for rotation
    private bool isGrounded;                                  // true when standing on the ground
    private bool jumpRequested = false;                      // set in Update(), used later
```

```

public bool IsGrounded => isGrounded;

private void Start()
{
    rb = GetComponent<Rigidbody>();
    rb.constraints = RigidbodyConstraints.FreezeRotation;
}

private void Update()
{
    // 1. Ground Check

    isGrounded = Physics.Raycast(transform.position +
transform.up*groundDistance/2, -transform.up, groundDistance, groundMask);

    // 2. Read Inputs

    turnInput = Input.GetAxisRaw("Horizontal"); // A/D or Left/Right
turn input

    float moveZ = Input.GetAxisRaw("Vertical"); // W/S or Up/Down
forward/backward

    // Rotate the player transform in Update for responsive turning
visuals

    transform.Rotate(0f, turnInput * turnSpeed * Time.deltaTime, 0f);

    // Calculate forward movement relative to current facing direction
moveDirection = transform.forward * moveZ;

    // 3. Handle Jump Input

    if (Input.GetButtonDown("Jump") && isGrounded)
    {

```

```
        jumpRequested = true;
    }
}

private void FixedUpdate()
{
    MovePlayer();

    if (jumpRequested)
    {
        Jump();
        jumpRequested = false;
    }
}

private void MovePlayer()
{
    // Calculate velocity based on current forward vector
    Vector3 targetVelocity = moveDirection * moveSpeed;

    // Apply movement while preserving vertical velocity
    (gravity/jumping)
    rb.linearVelocity = new Vector3(targetVelocity.x,
    rb.linearVelocity.y, targetVelocity.z);
}

private void Jump()
{

```

```

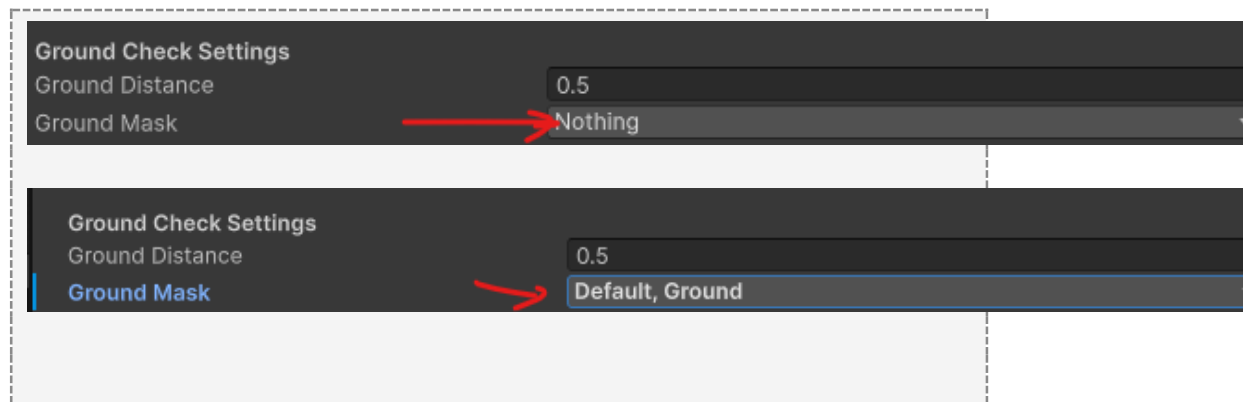
        rb.linearVelocity = new Vector3(rb.linearVelocity.x, 0f,
rb.linearVelocity.z);

        rb.AddForce(Vector3.up * jumpForce, ForceMode.VelocityChange);
    }
}

```

10. Configure the script in the Inspector:

- Return to Unity and wait for the script to compile. Select the Player and find the **Player Movement** component.
- Set **Ground Mask** to **Ground and Default**. If you skip this, the raycast will never hit anything and the player won't be able to jump.
- Leave **Move Speed** at 7, **Jump Force** at 5 and **Ground Distance** at 0.5 for now. You can tune these later.
- Notice that a **Rigidbody** component has appeared automatically thanks to `[RequireComponent]`.



11. Test the movement:

- Press **Play**. Move with **WASD** or the arrow keys and jump with **Space**.
- The character will slide around in a single pose. That is expected: nothing is telling the Animator what to play yet. Fixing that is what the rest of this lab is about.
- **Can't jump?** Check that the floor is on the Ground layer, Ground Mask is set to Ground, and the floor has a collider. If your character's pivot is at its centre rather than its feet, the ray may be too short: increase **Ground Distance** (about 1.1).
- **Falling through the floor?** The floor or the Player is missing a collider.
- **Character tips over?** Make sure the Rigidbody's rotation constraints are frozen (the script does this in `Start()`).

12. Check your understanding:

- Why do we store the jump in `jumpRequested` instead of calling `Jump()` directly inside `Update()`? :

Because physics related movements should be stored in `FixedUpdate()`, while key strokes should be caught in `Update()`.

-
- What would happen to gravity if `MovePlayer()` set the velocity to `new Vector3(targetVelocity.x, 0f, targetVelocity.z)`?

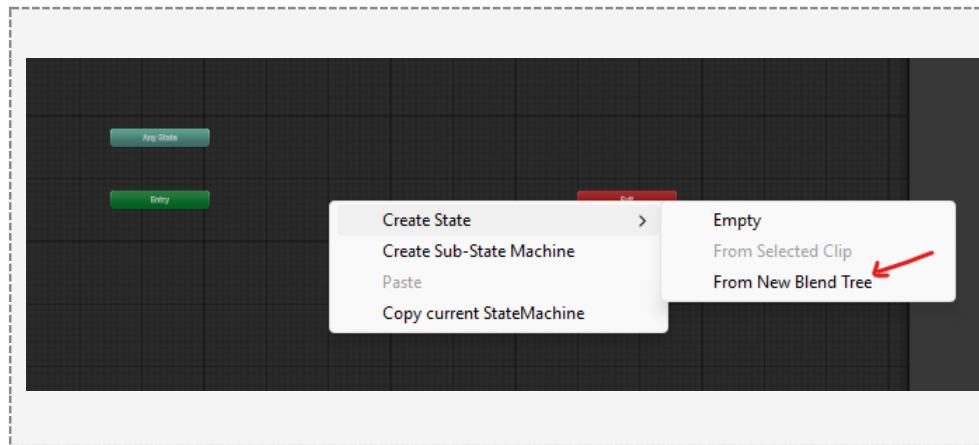
Gravity would simply not work.

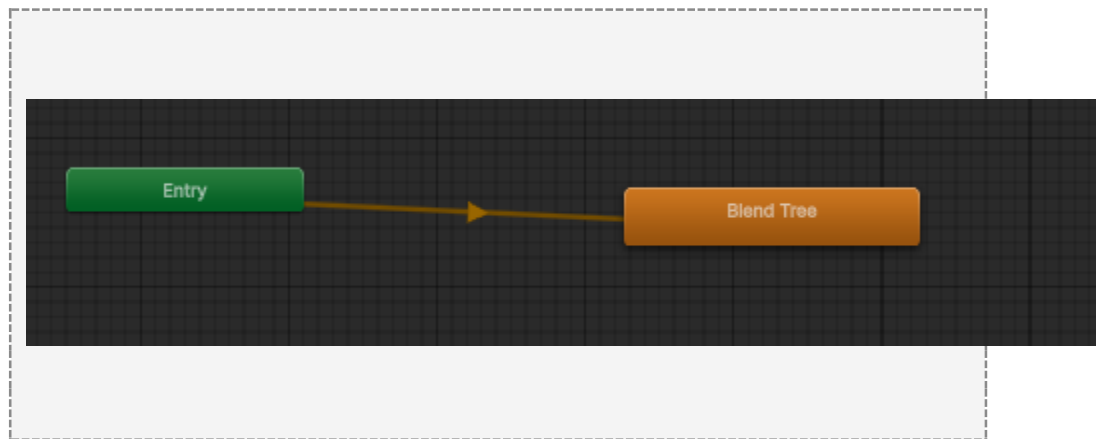
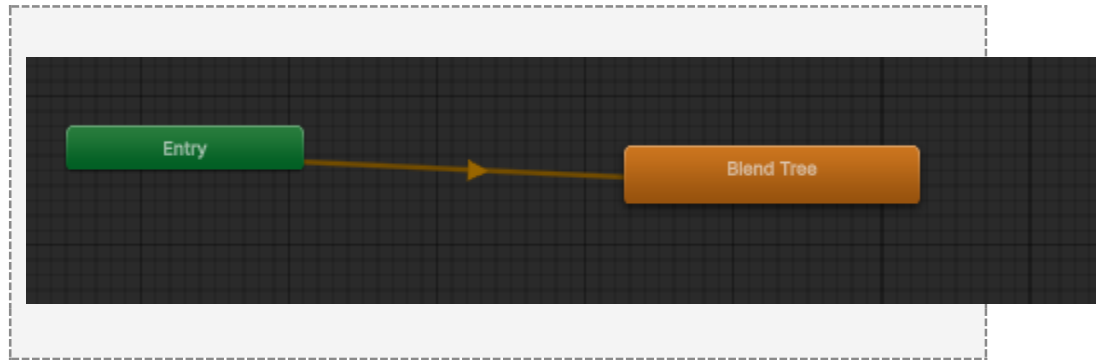
- What happens to diagonal movement speed if you remove `.normalized`? Try it and see.

Task 3: Create a Blend Tree for Movement (Idle/Walking/Running)

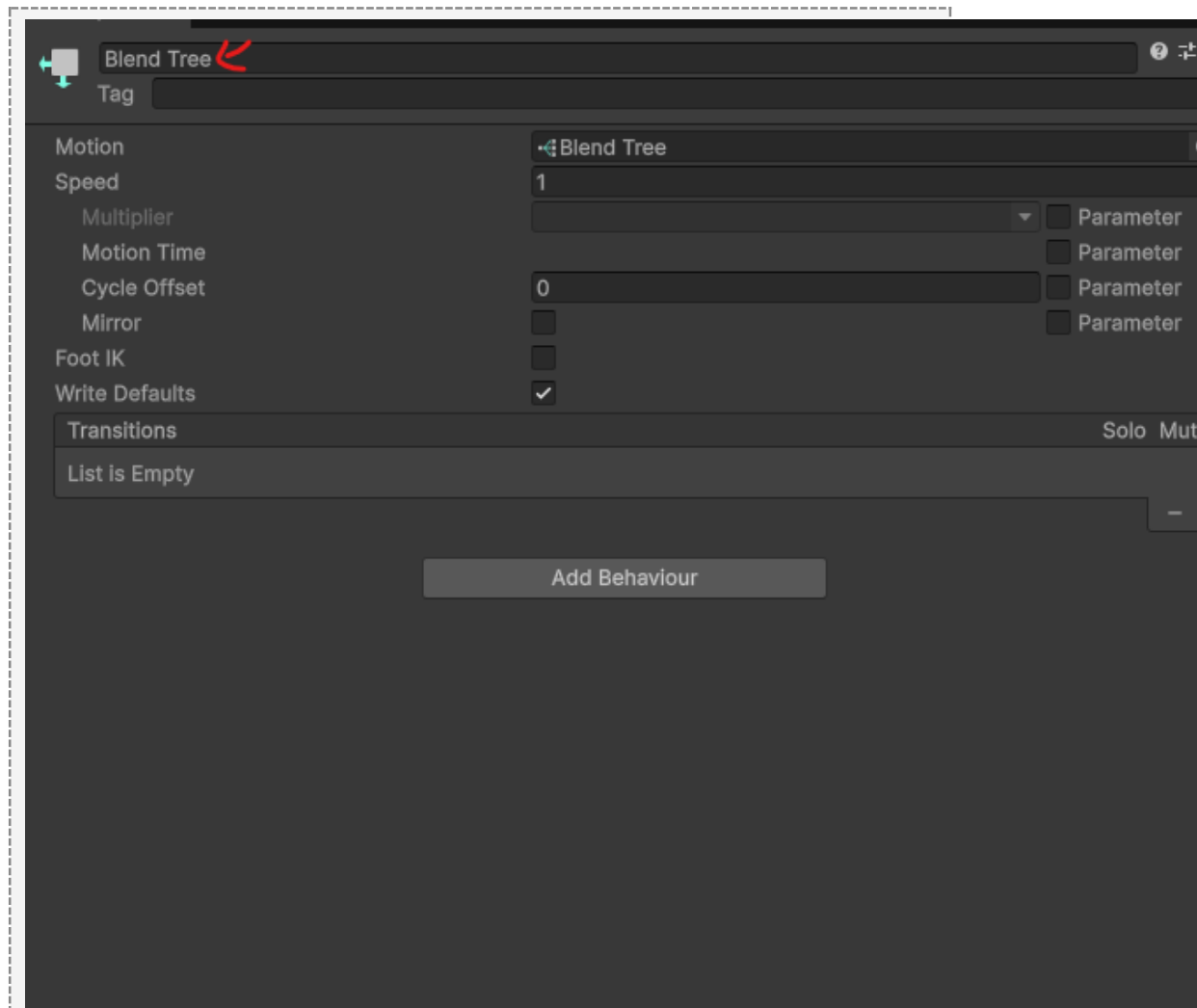
1. Create a Blend Tree State:

- In the **Animator** window, right-click in an empty space and choose **Create State > From New Blend Tree**.





- In order to rename it. Right click on the Orange box and details of that stat should show up in the inspector



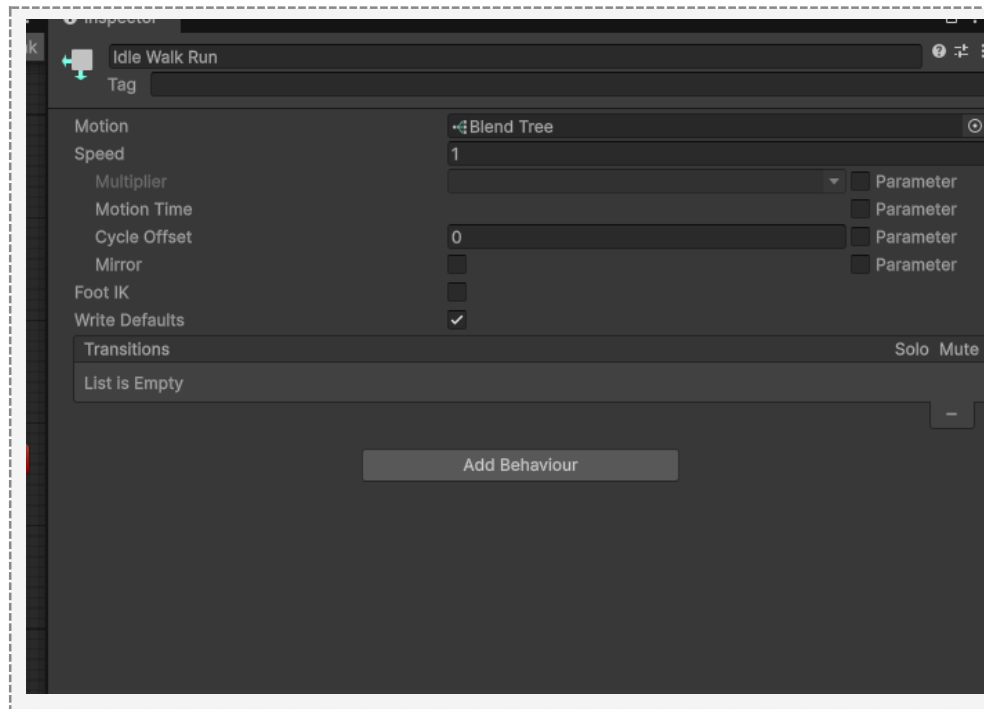
[Screenshot]

Inspector showing the selected state's name field

- You can now rename the State at the top of the Inspector
- Rename this blend tree **Idle/Walking/Running** to represent the different movement states.
- By default, this state should appear in **orange**, meaning it is the entry state (the default state when the game starts).

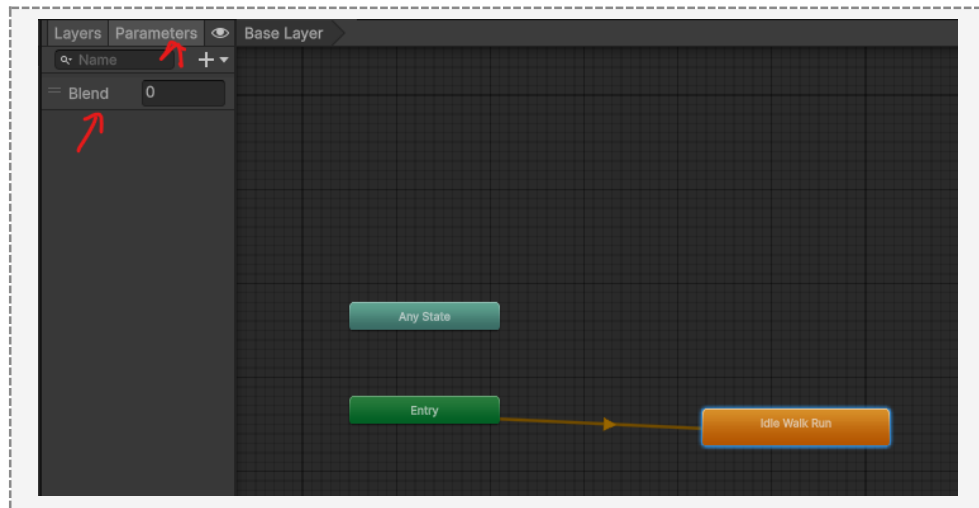
2. Open the Blend Tree:

- Double-click on the **Idle/Walking/Running** state to open the blend tree editor.



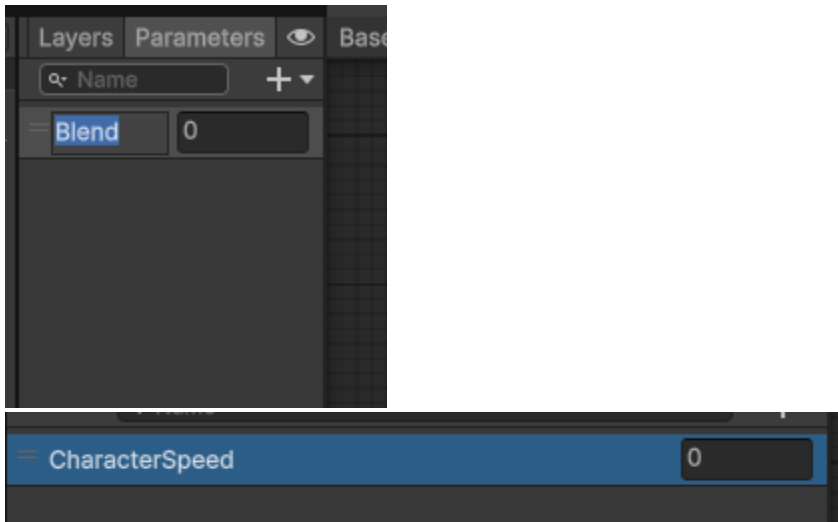
3. Set the Blend Parameter:

- In the **Inspector** panel (while the blend tree is selected), you will see a **Blend** section.



-
-

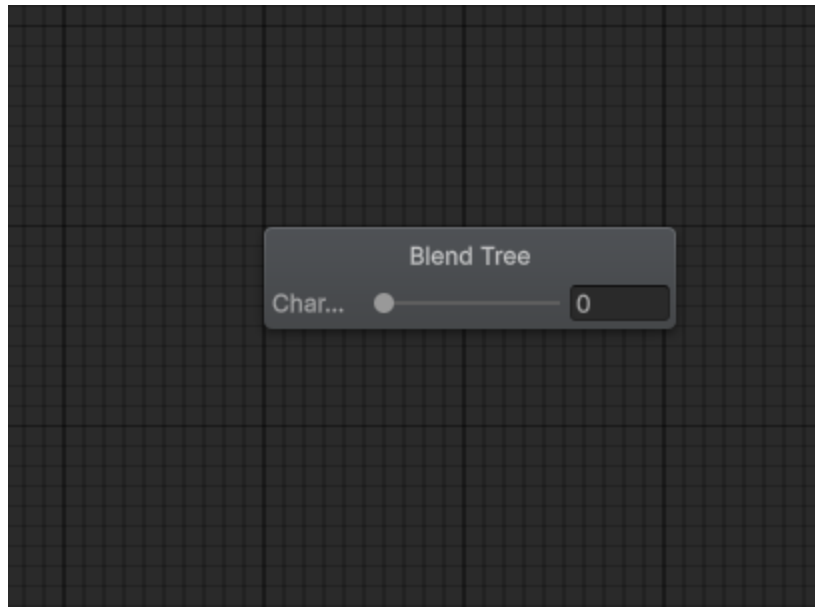
- Set the **Blend Parameter** to a new parameter called **CharacterSpeed**. By double clicking on the Blend parameter This parameter will control which animation (Idle, Walking, or Running) plays based on the character's speed.



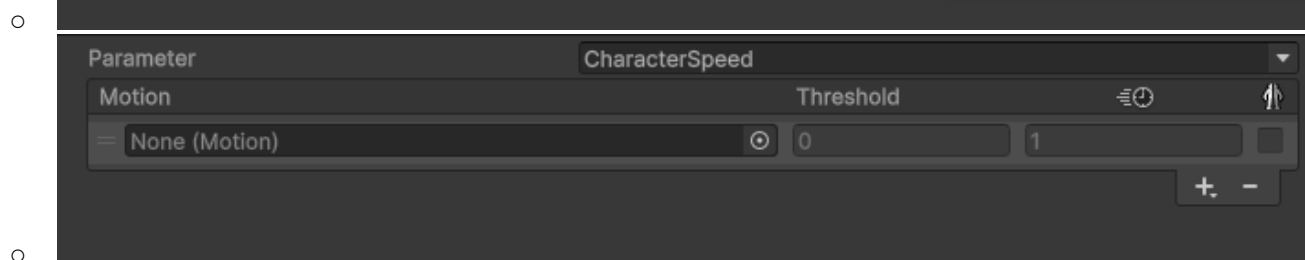
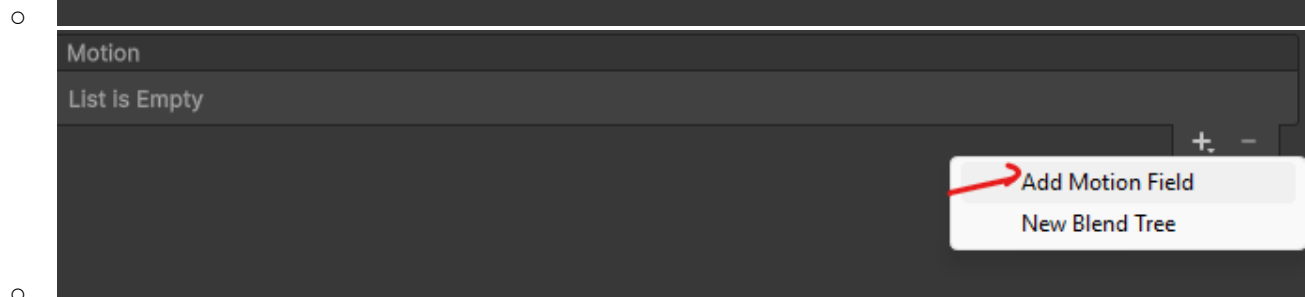
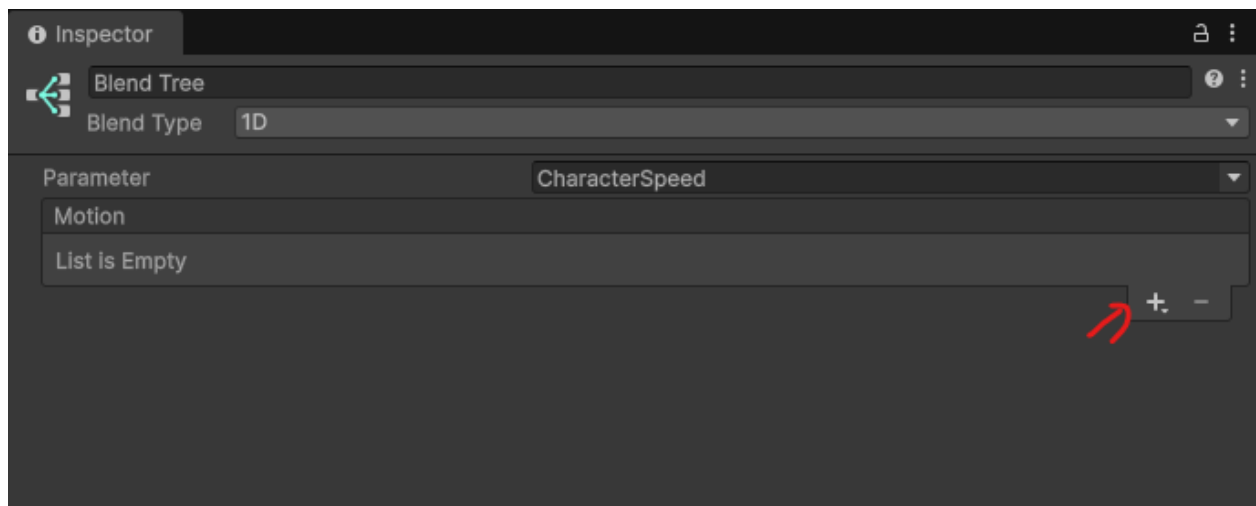
4. Add Animations to the Blend Tree:

- Double click on the Blend Tree State



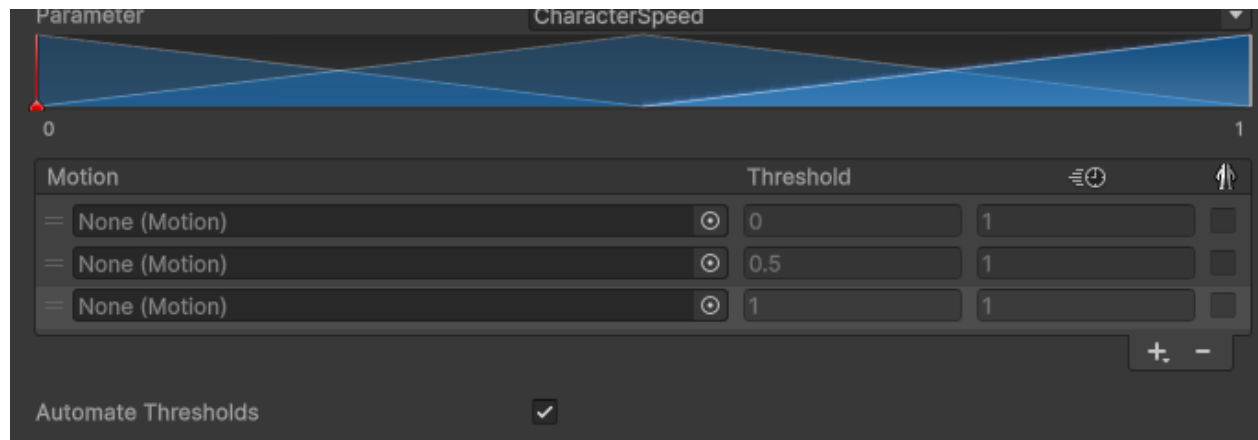


-
- In the **Inspector** panel, click **+ Add Motion Field** to add animation slots to the blend tree.

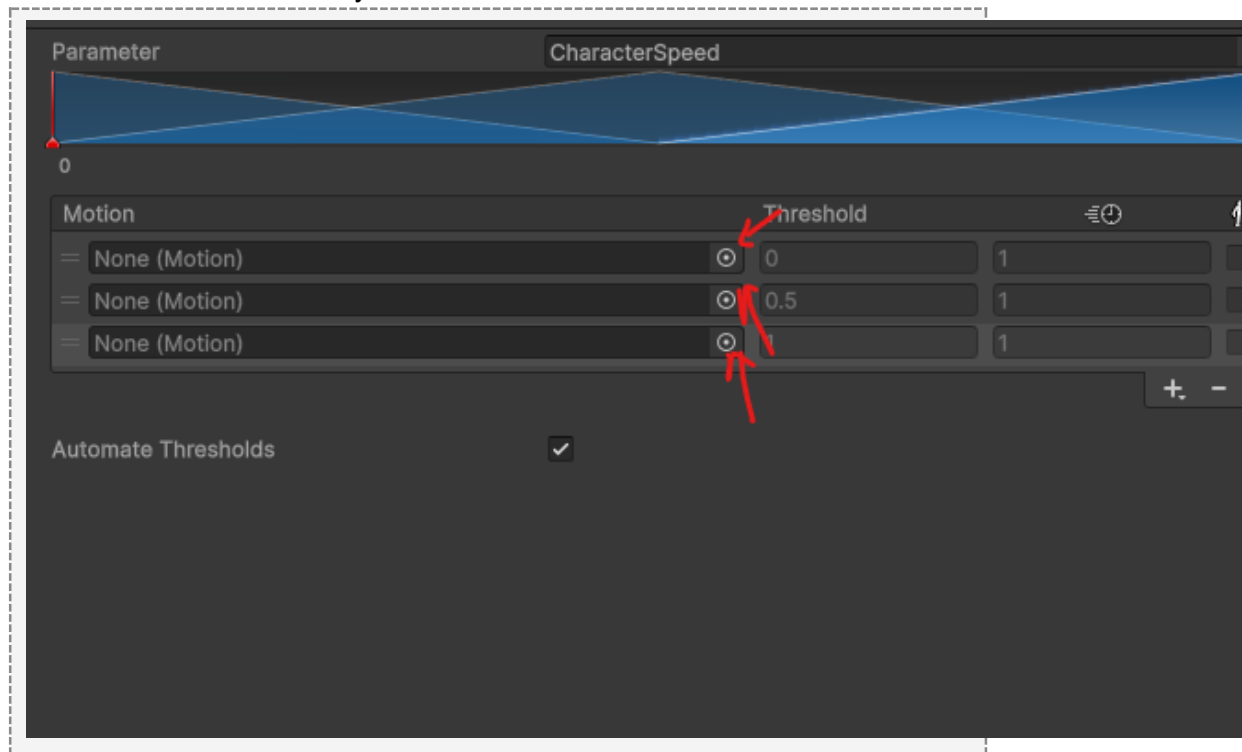


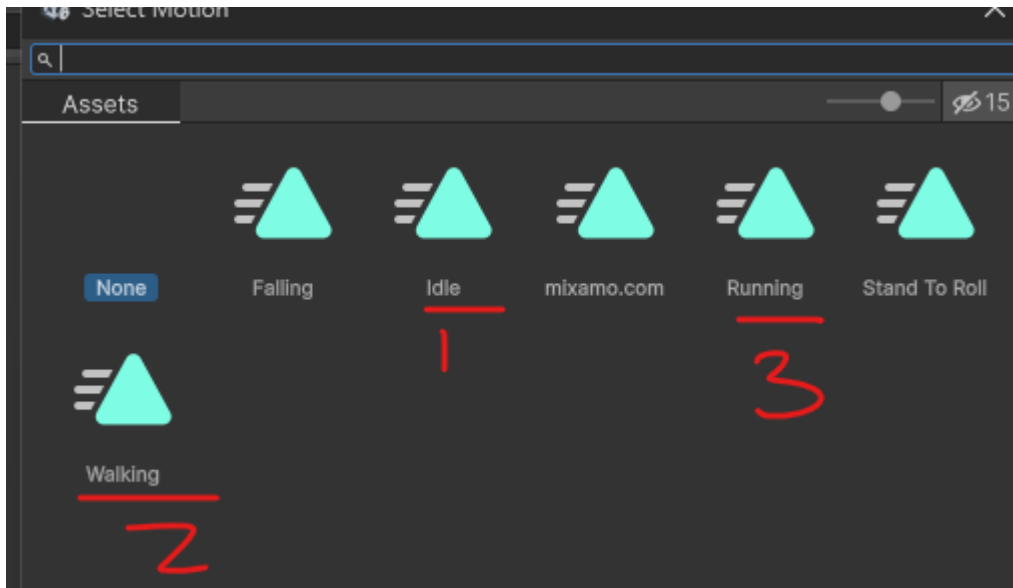
-

- Do that 3 times

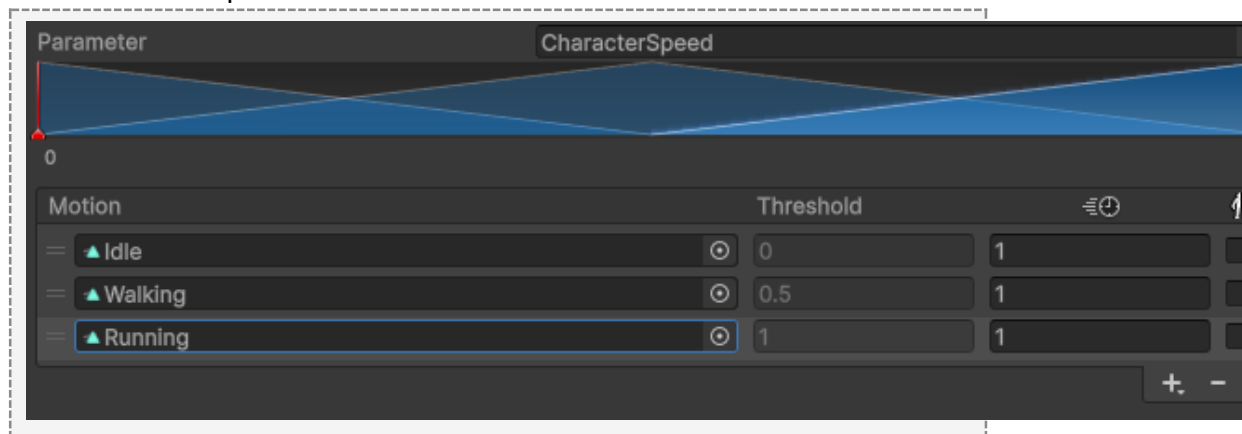


- Click the little dot in the entry of the List of Motion, then select



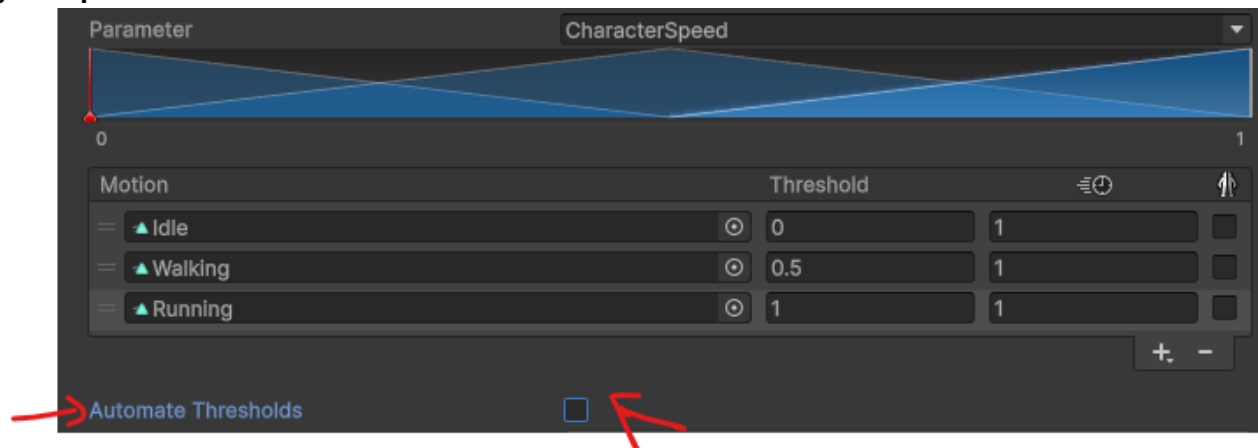


-
- **Idle** to the first slot , **Walking** to the second, and **Running** to the third slot.
- Uncheck the Compute threshold checkbox.



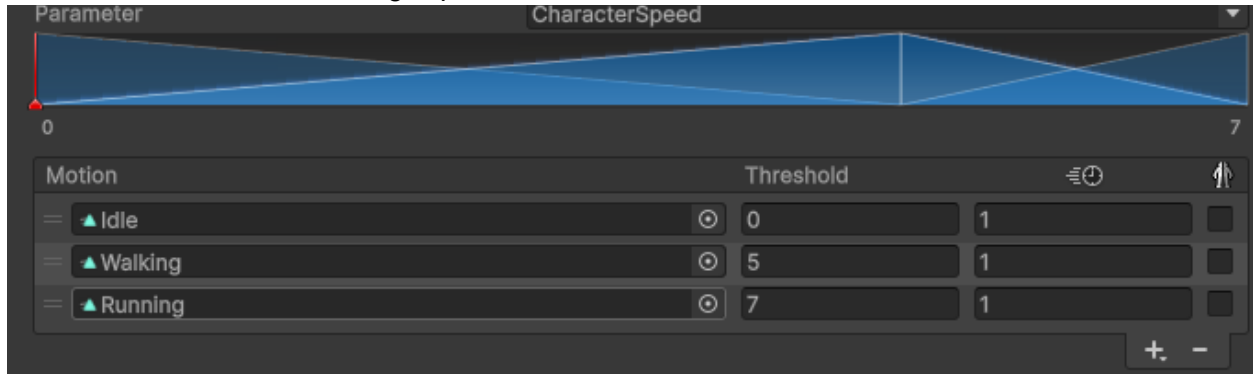
- Your blend tree all setup should look like the picture above

5. Configure Speed Thresholds:



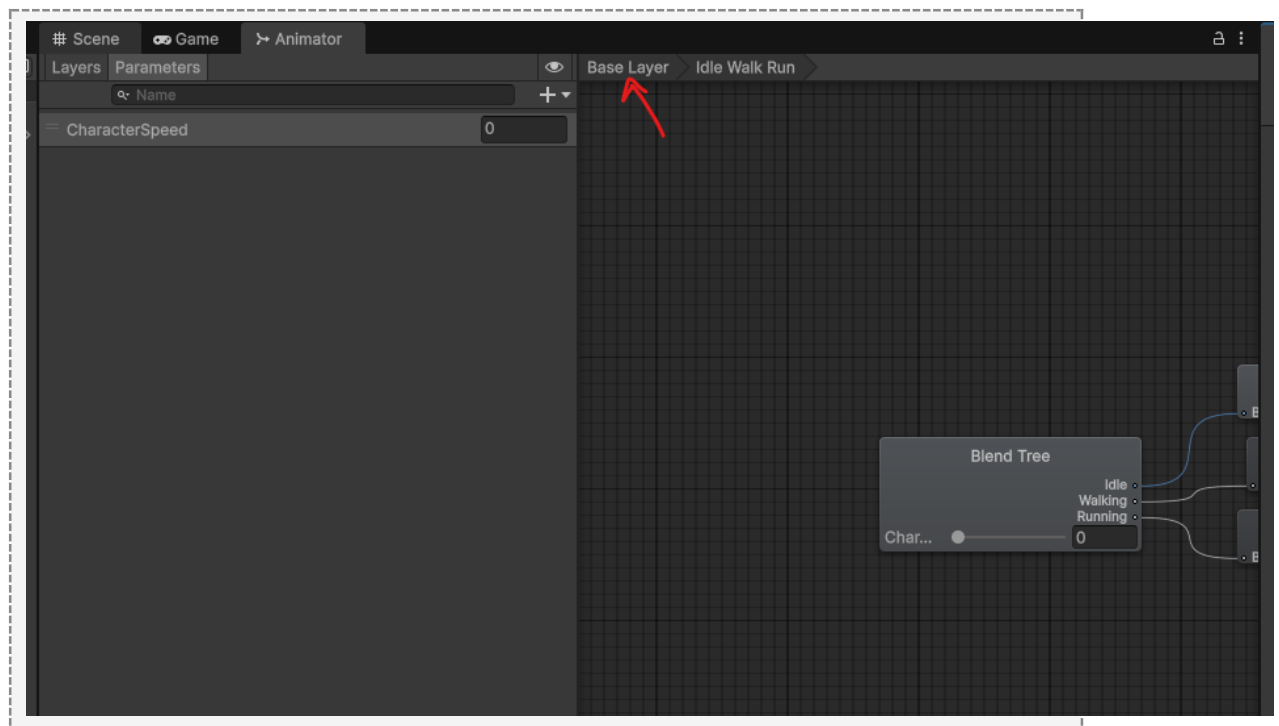
-

- Set the speed values for each animation to correspond with different character speeds:
 - Idle: Speed = 0
 - Walking: Speed = 5
 - Running: Speed = 7



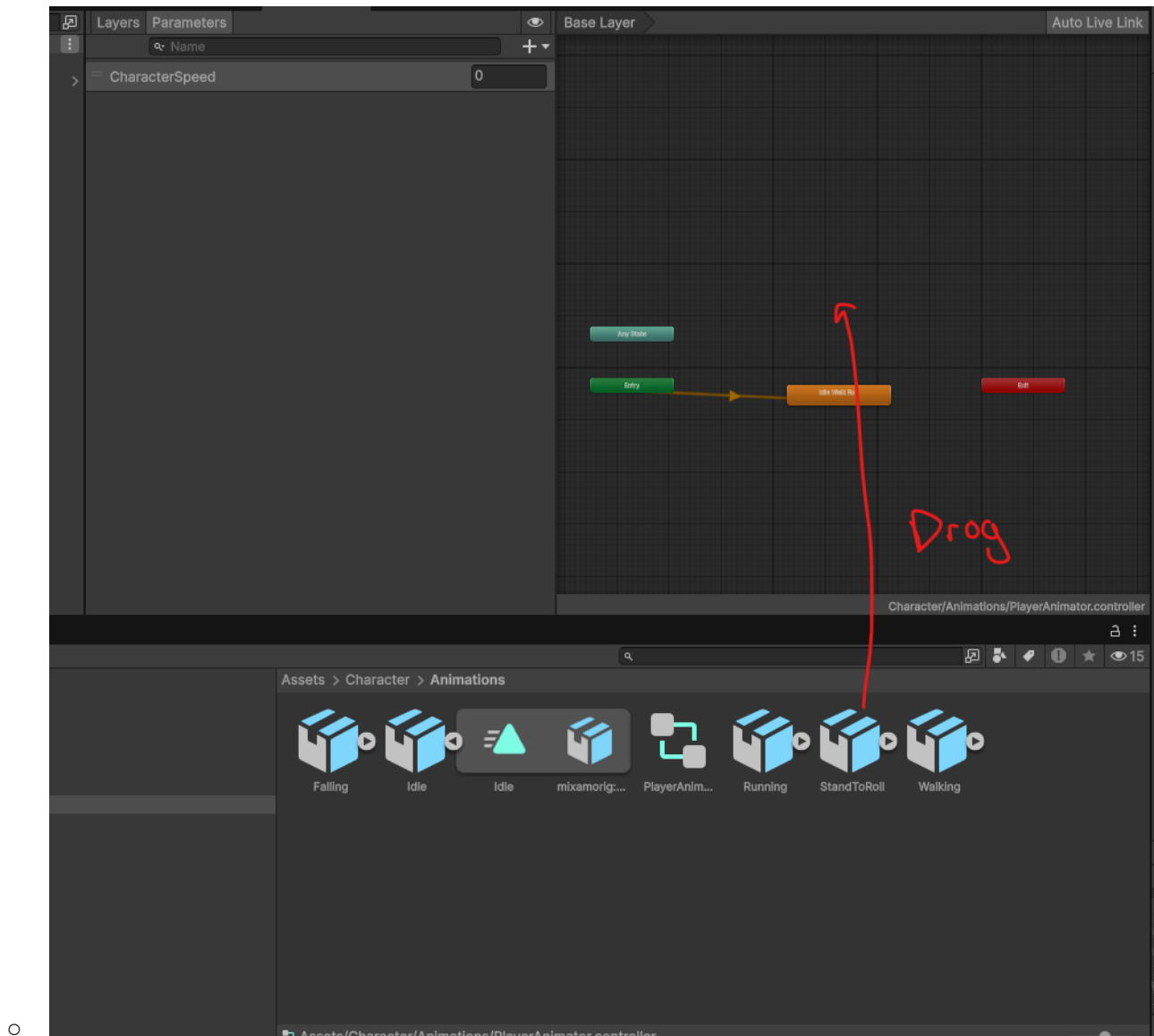
- As the **CharacterSpeed** parameter increases from 0 to 7, the blend tree will interpolate between these animations based on the speed of the character. These thresholds line up with the **moveSpeed** of 7 you set in **PlayerMovement** in Task 2, so the character reaches the full Running animation at top speed.

Go back to the base layer in the Animator by clicking on the Base Layer Button :

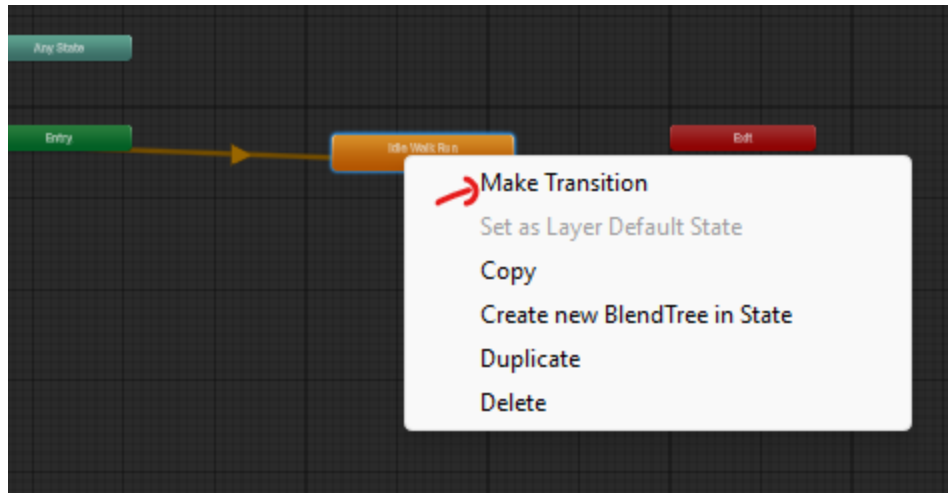


Task 4: Create Transitions for the "Stand to Roll" Animation

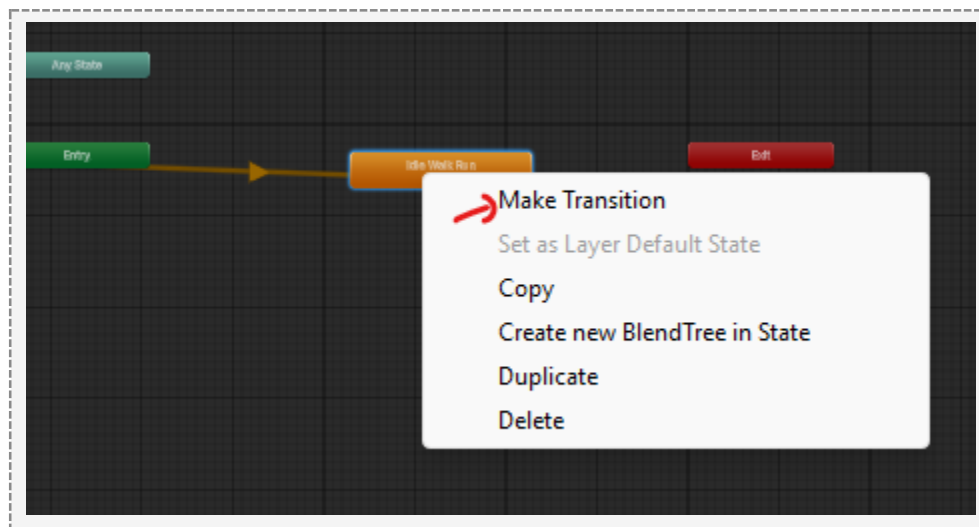
1. **Add the Rolling Animation:**
 - Drag the **Stand to Roll** animation from your **Assets** folder into the **Animator** window, creating a new state.



2. **Create a Transition from Idle/Walking/Running to Rolling:**
 - Right-click on the Idle/Walking/Running blend tree and select **Make Transition**.

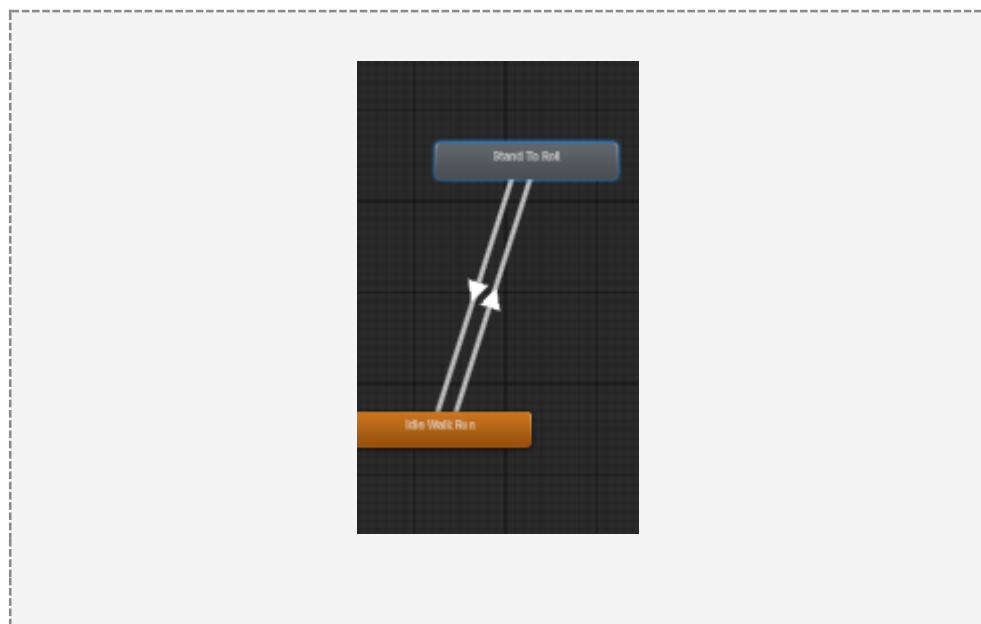
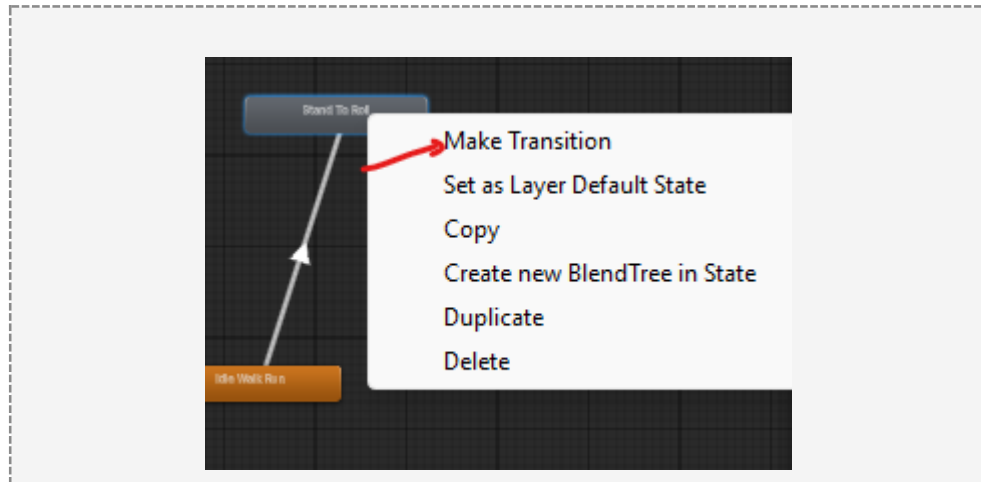


-
- Drag the arrow to the **Stand to Roll** state.



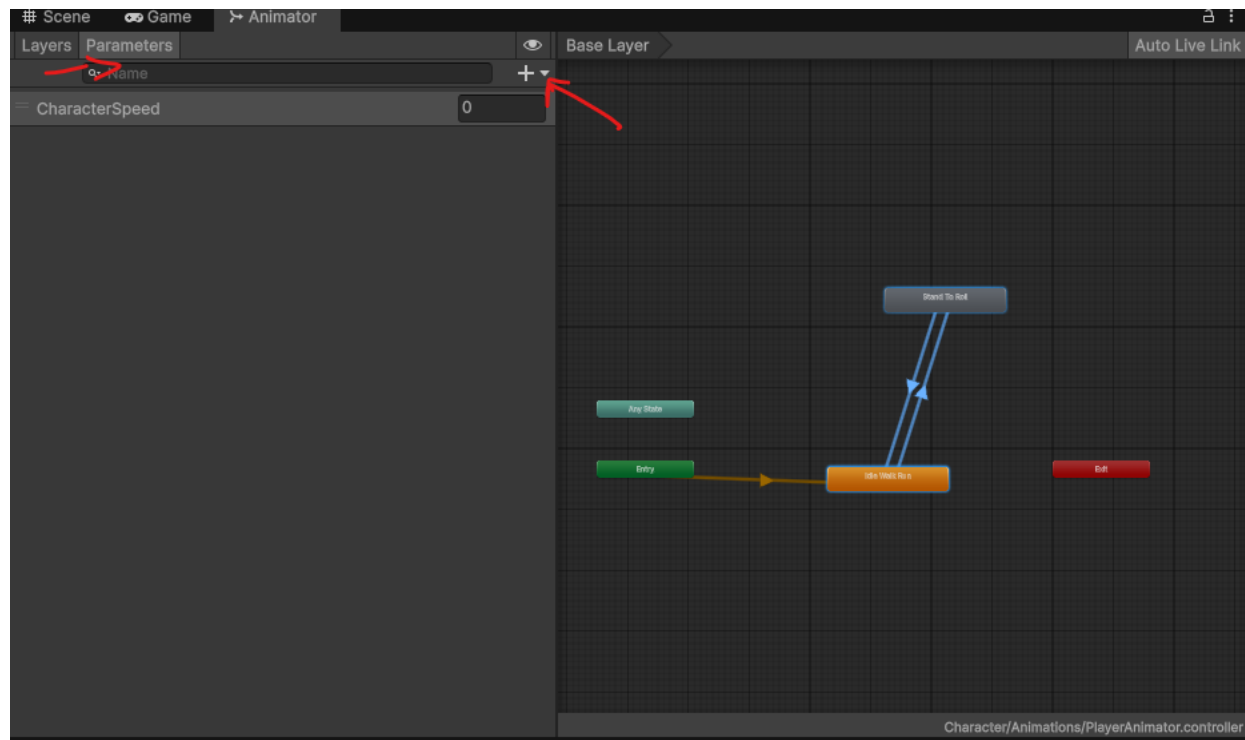
3. Create a Transition from Rolling back to **Idle/Walking/Running**:

- Right-click on the **Stand to Roll** state, choose **Make Transition**, and drag the arrow back to the **Idle/Walking/Running** blend tree.

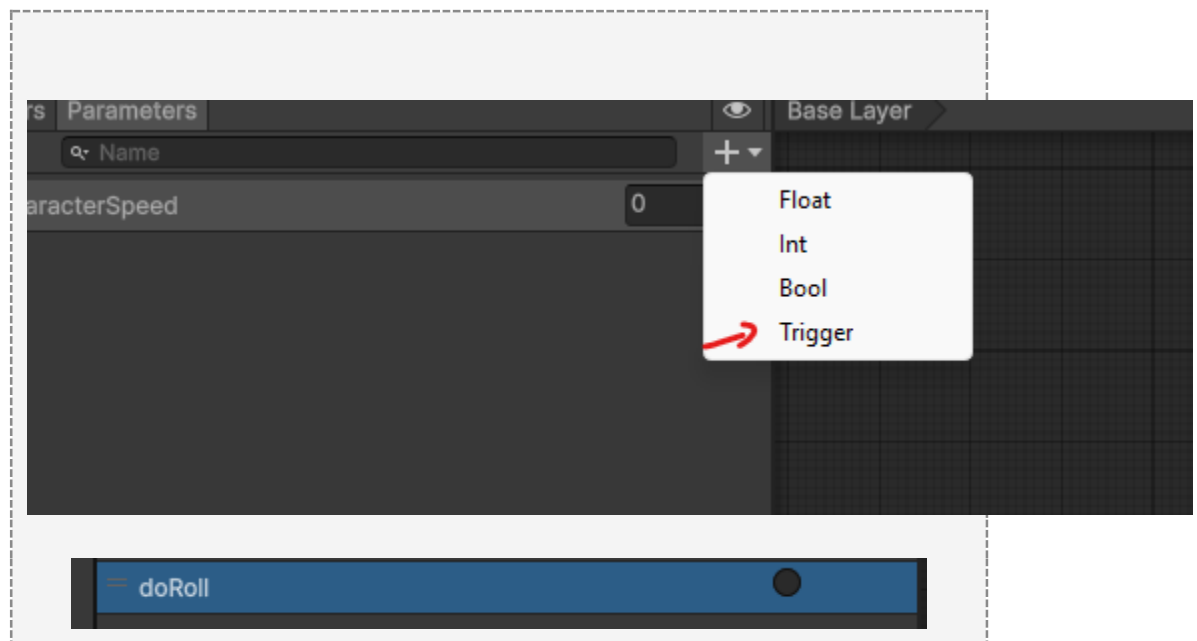


4. **Setup the "IsRolling" Parameter:**

- In the **Animator** window, click the **Parameters** tab (next to Layers).



-
- Add a new **Trigger** parameter called `doRoll`.

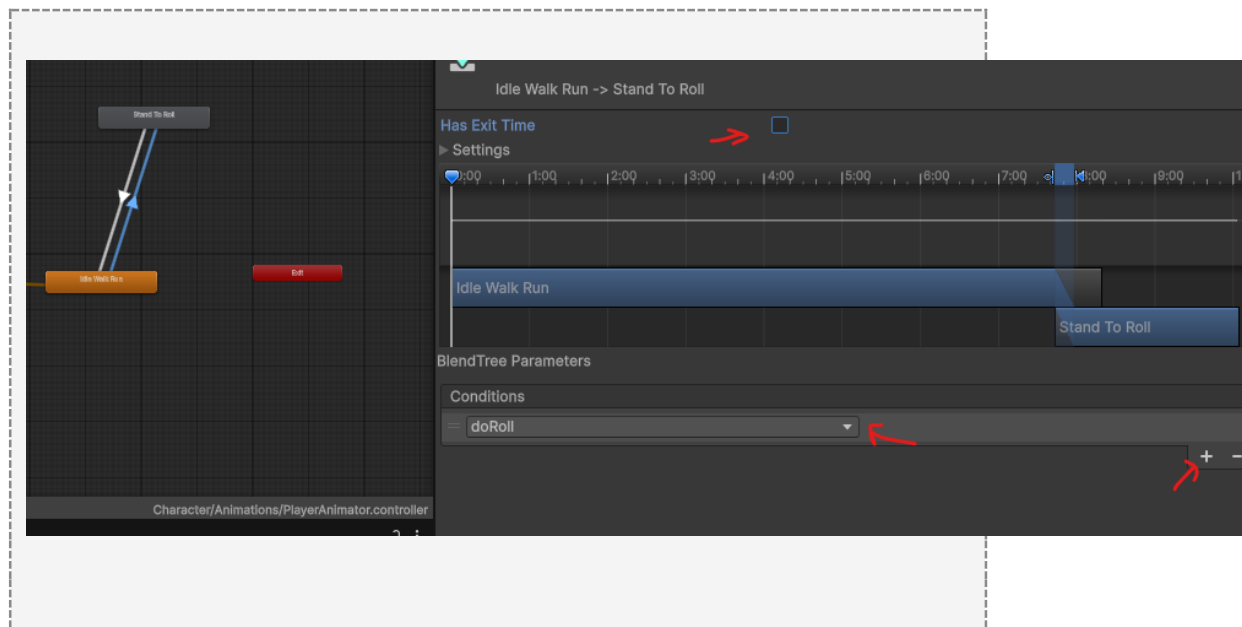




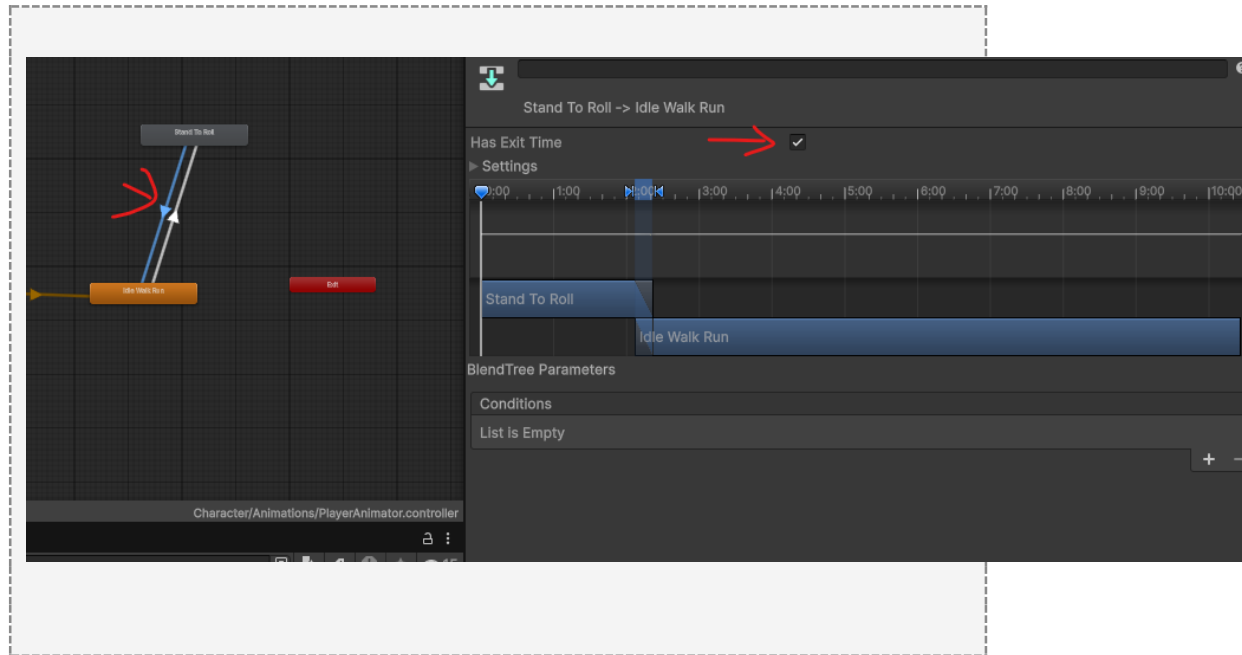
- By the end your stand to roll should be connected like the previous picture

5. Configure Transition Conditions:

- Select the transition from **Idle/Walking/Running** to **Stand to Roll**.
 - Add the parameter doRoll in the condition
 - Uncheck Has Exit time

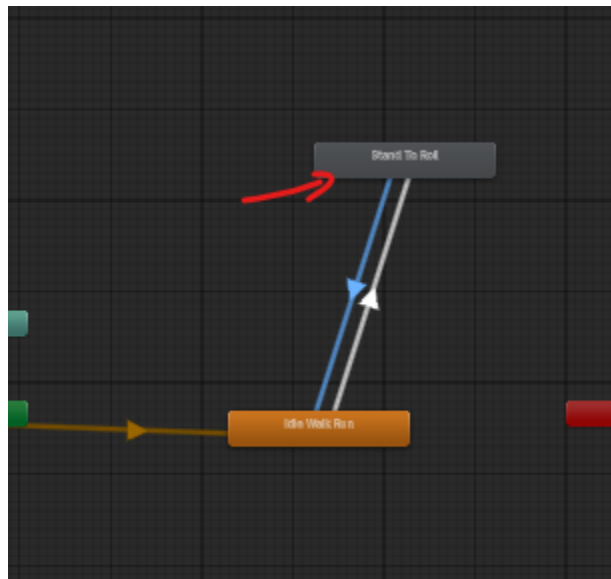


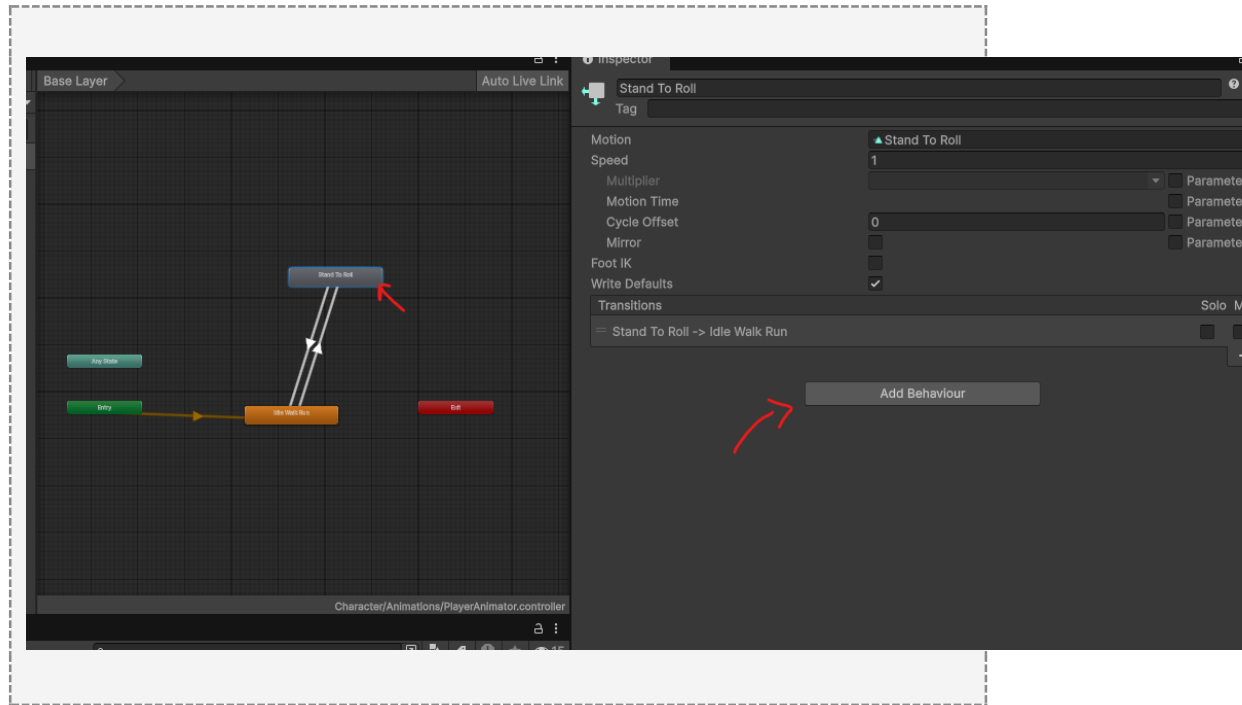
- Select the transition back to **Idle/Walking/Running** make sure the HasExitTime is still checked to make sure the roll animation plays fully before going back to the Idle/Walk/Run state



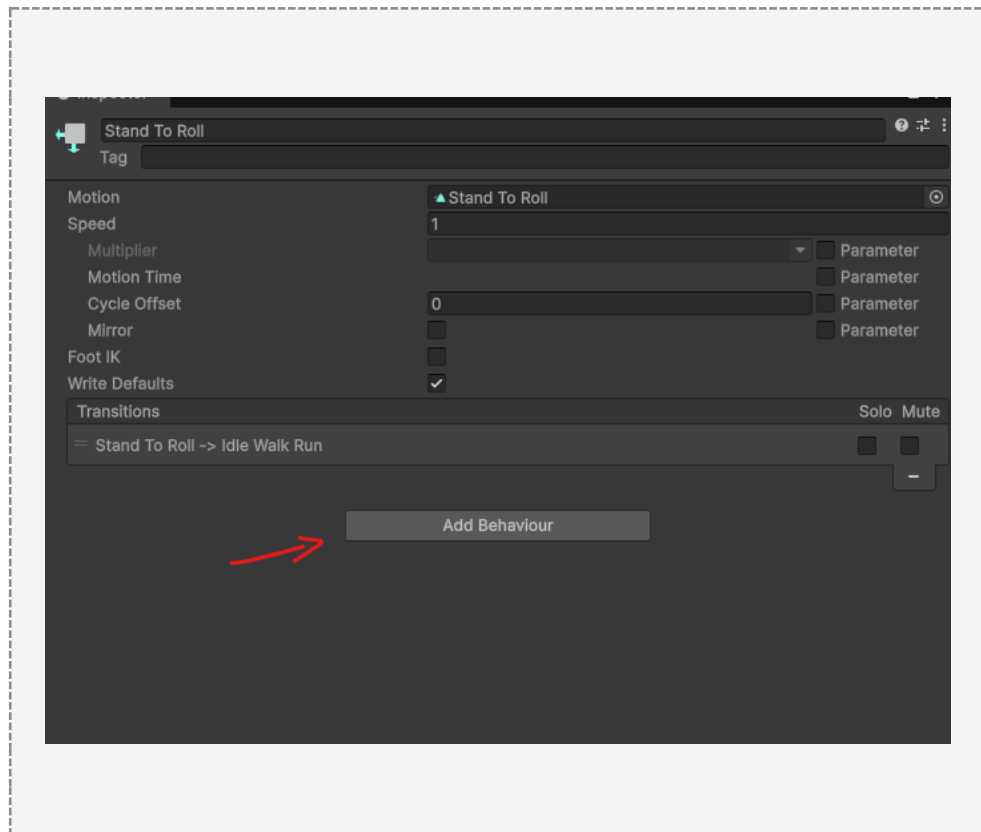
Adding a behaviour to the roll.

Click on the state Stand to roll

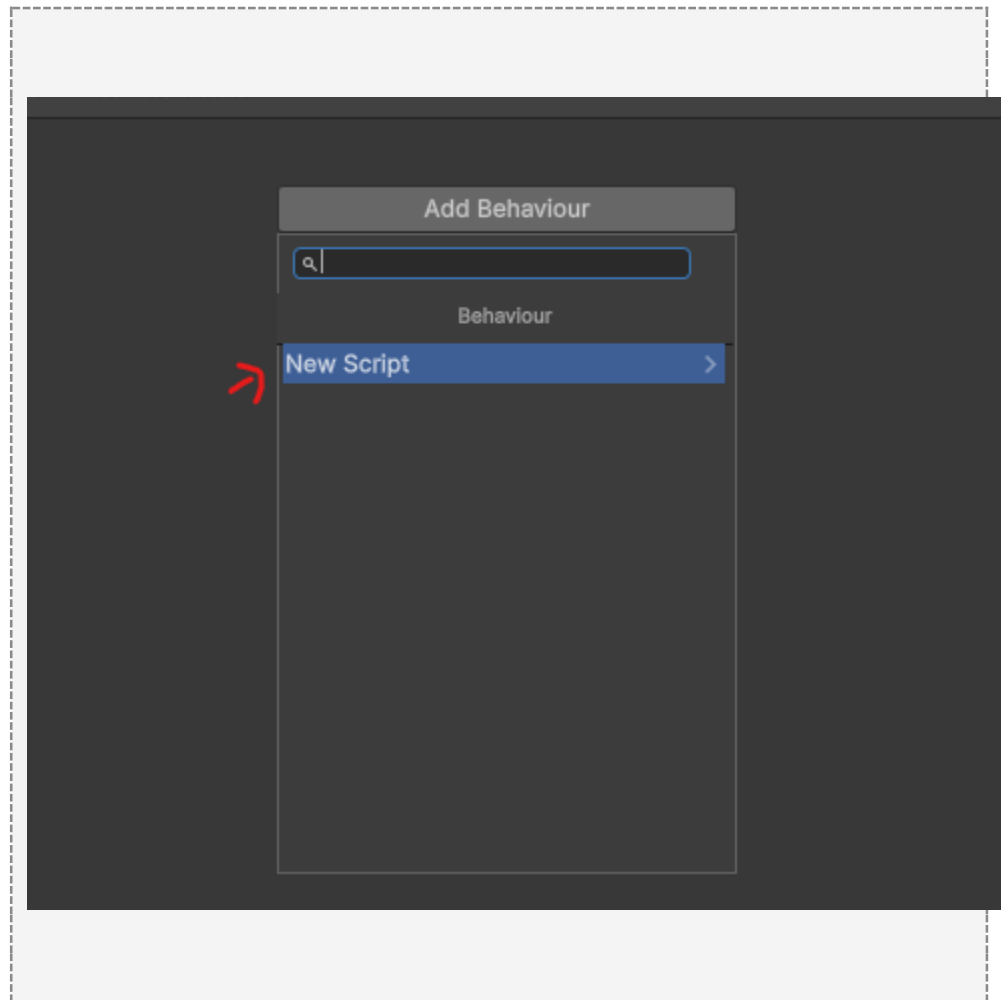




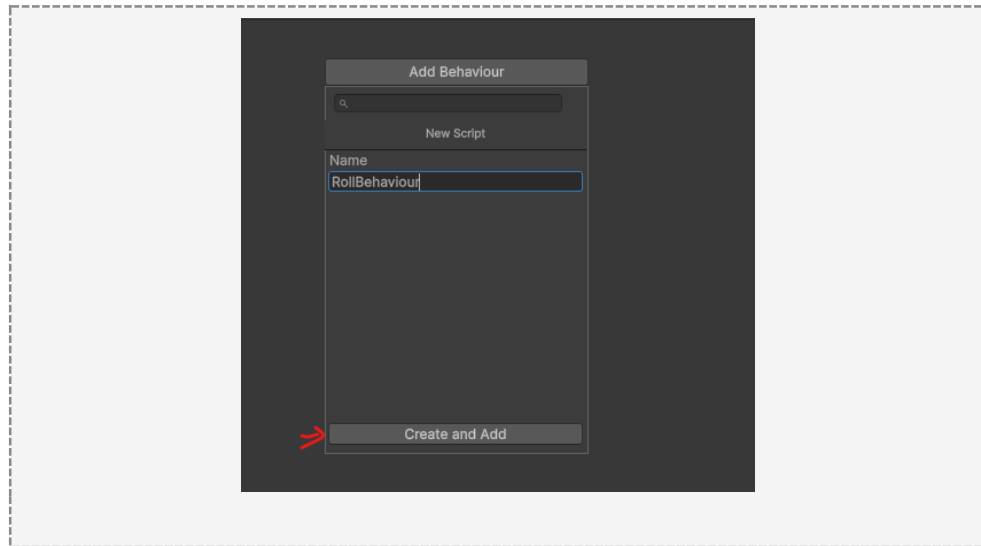
On the left panel you should see this window click on the Add behaviour button.



Click new Script



Call it **Roll Behaviour**



Update the script you just created this script. This will enable and disable rootmotion at the beginning and end of the roll animation so that only the roll is affected by root motion.

```
using UnityEngine;

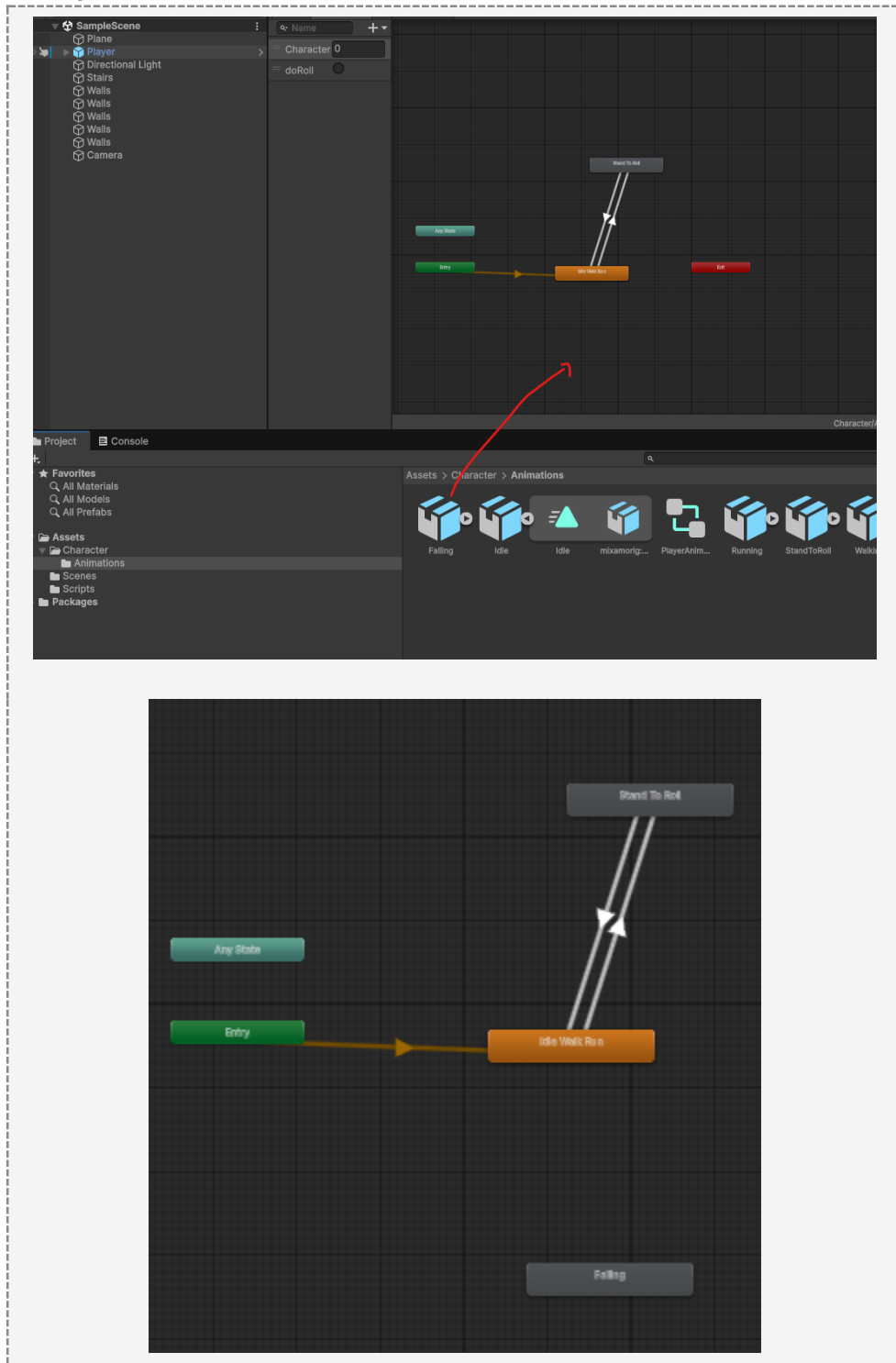
// A StateMachineBehaviour runs on an Animator state,
// not on a GameObject.
// This one is attached to the Stand to Roll state.
public class RollBehaviour : StateMachineBehaviour
{
    // Called when the Animator starts playing this state
    public override void OnStateEnter(Animator animator,
        AnimatorStateInfo stateInfo, int layerIndex)
    {
        // Let the roll animation itself move the character forward
        animator.applyRootMotion = true;
    }

    // Called when the Animator finishes this state
    public override void OnStateExit(Animator animator,
        AnimatorStateInfo stateInfo, int layerIndex)
    {
        // Hand movement back to the PlayerMovement script
        animator.applyRootMotion = false;
    }
}
```

Task 5: Create Transitions for Falling Animation

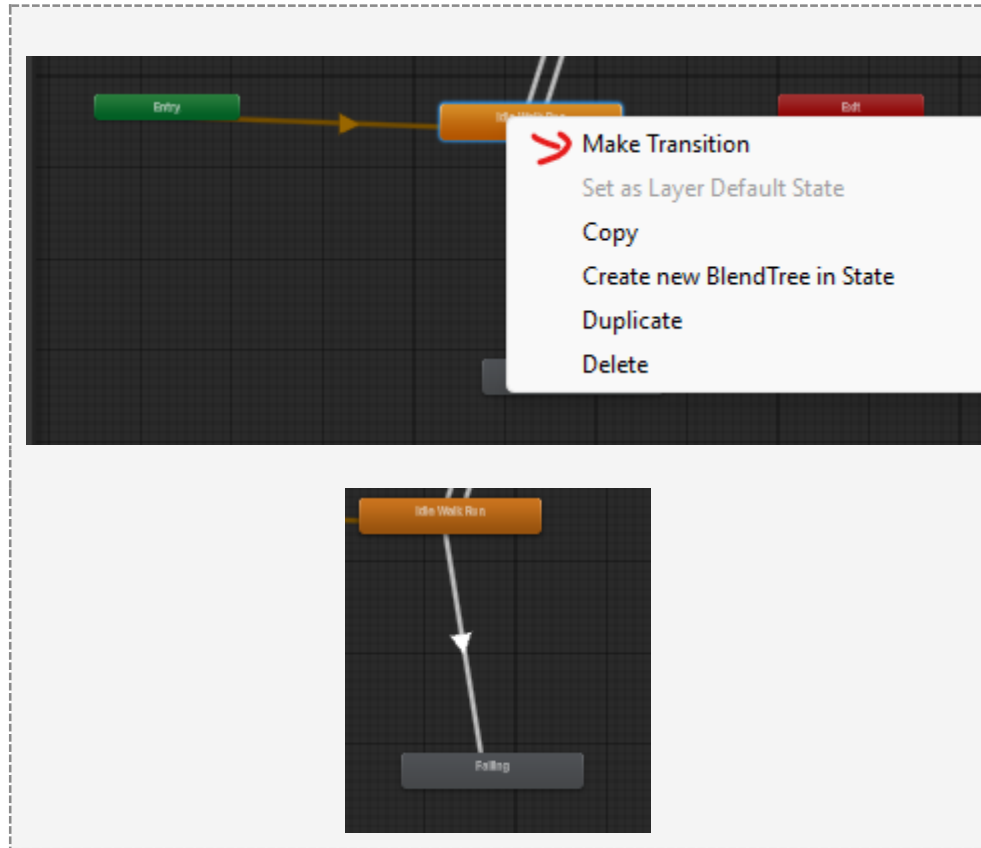
1. Add the Falling Idle Animation:

- Drag the **Falling** animation from your **Assets** folder into the **Animator** window, creating a new state.

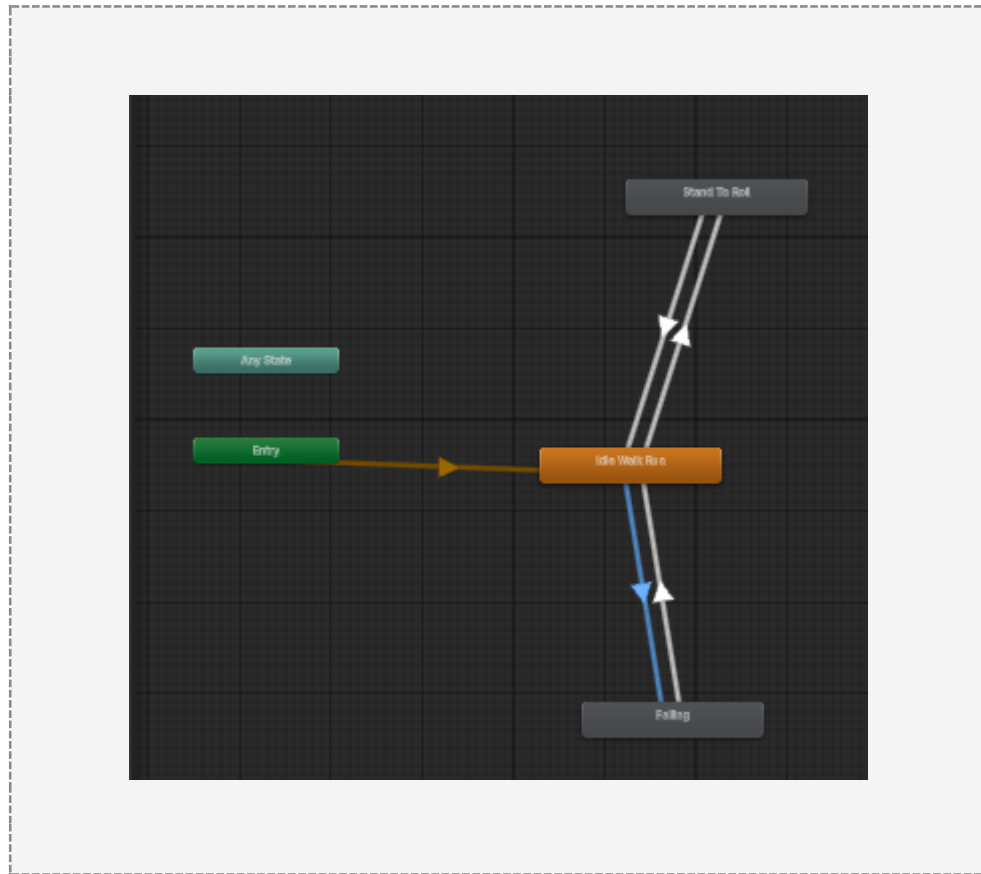


2. Create a Transition from **Idle/Walking/Running** to **Falling**:

- Right-click on the **Idle/Walking/Running** state, select **Make Transition**, and drag the arrow to the **Falling** state.

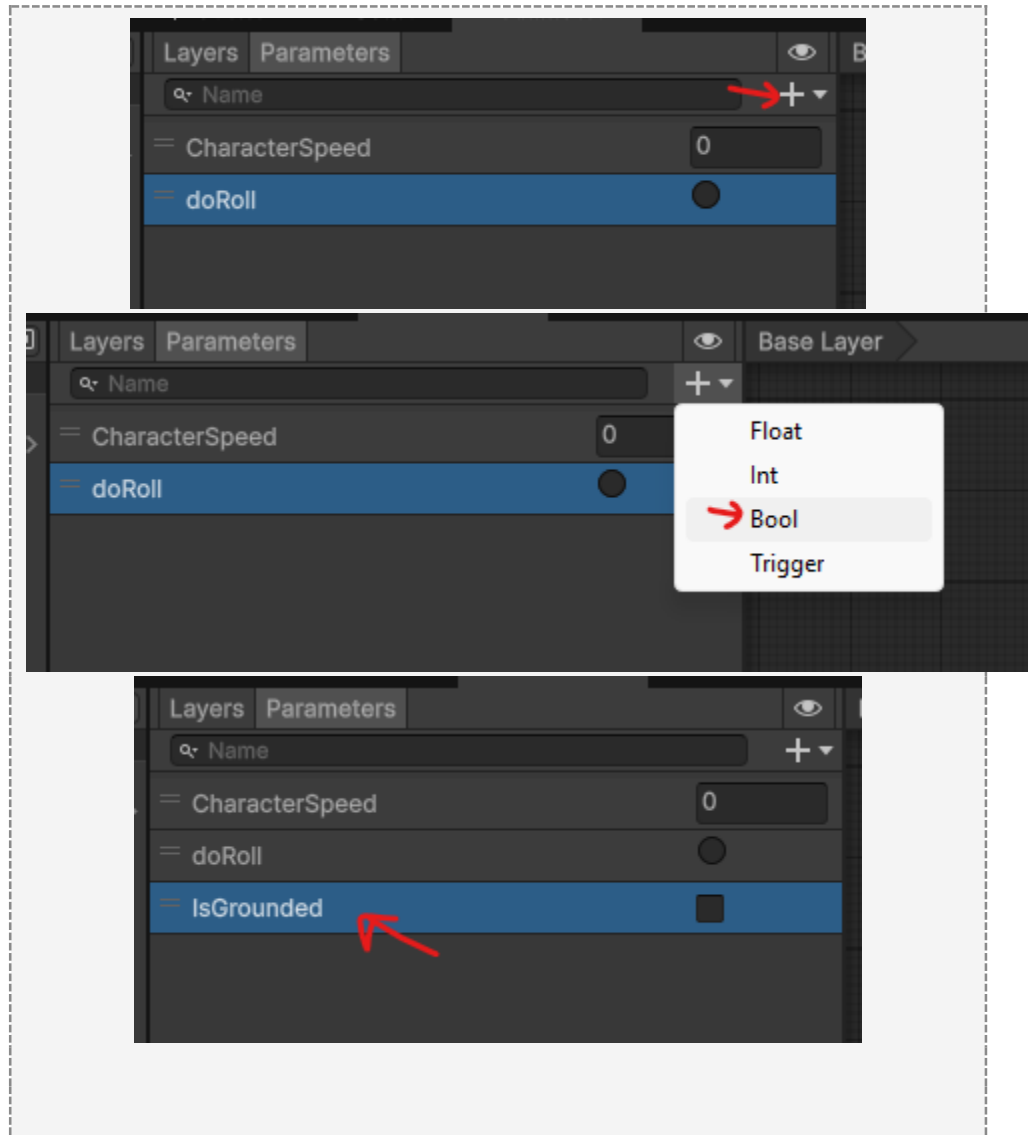


3. **Create a Transition from Falling back to Idle/Walking/Running:**
 - Right-click on the **Falling** state, select **Make Transition**, and drag the arrow back to **Idle/Walking/Running**.



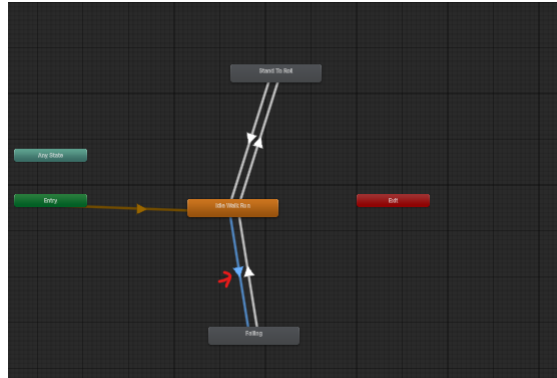
4. **Setup the "IsGrounded" Parameter:**

- In the **Parameters** tab, add a new **Bool** parameter called **IsGrounded**.

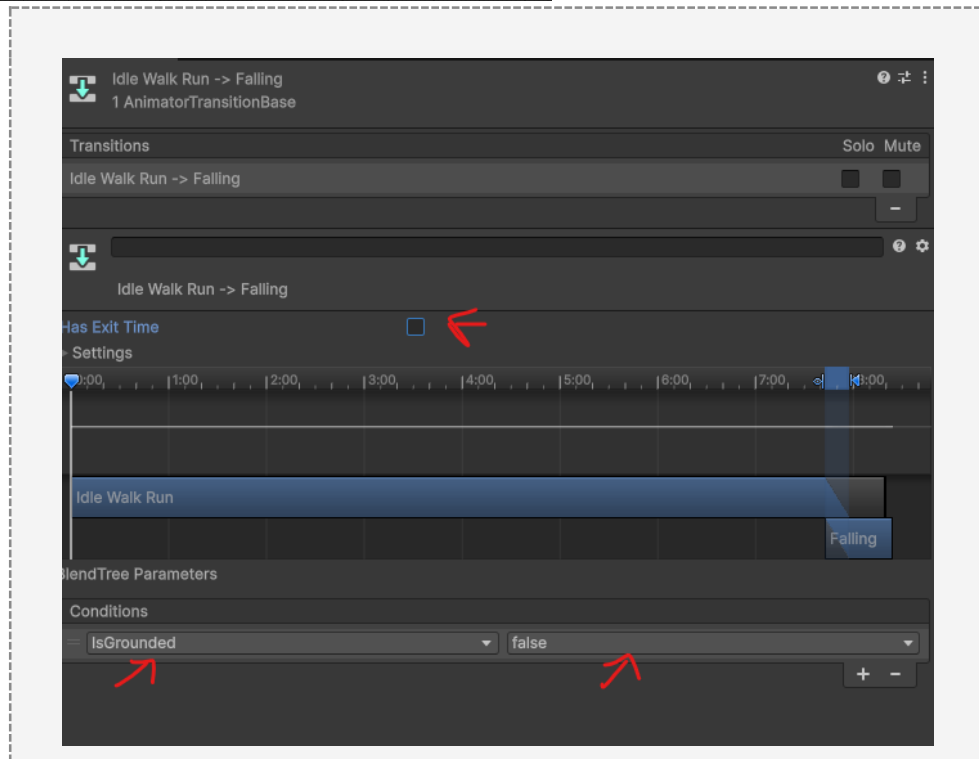


5. Configure Transition Conditions:

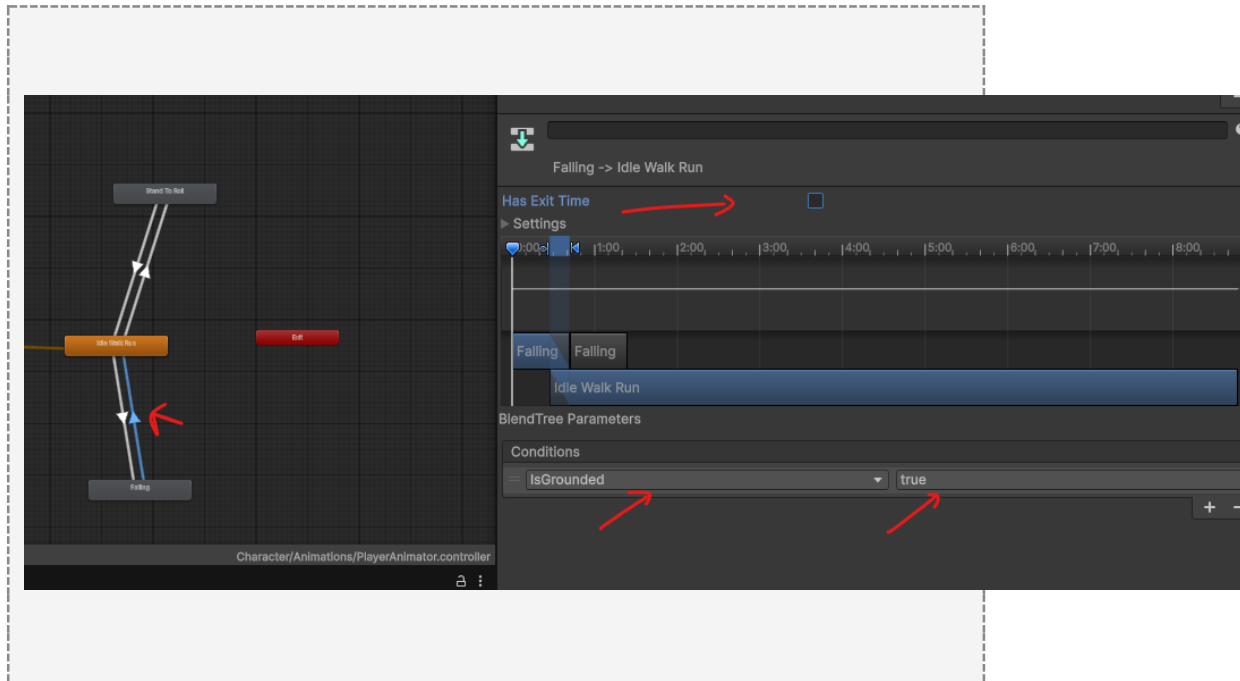
- Select the transition from **Idle/Walking/Running** to **Falling**.
- In the **Inspector**, set the condition to **IsGrounded== false** and make sure **HasExitTime** is checked off. (We want to transition as soon as IsFalling Becomes true



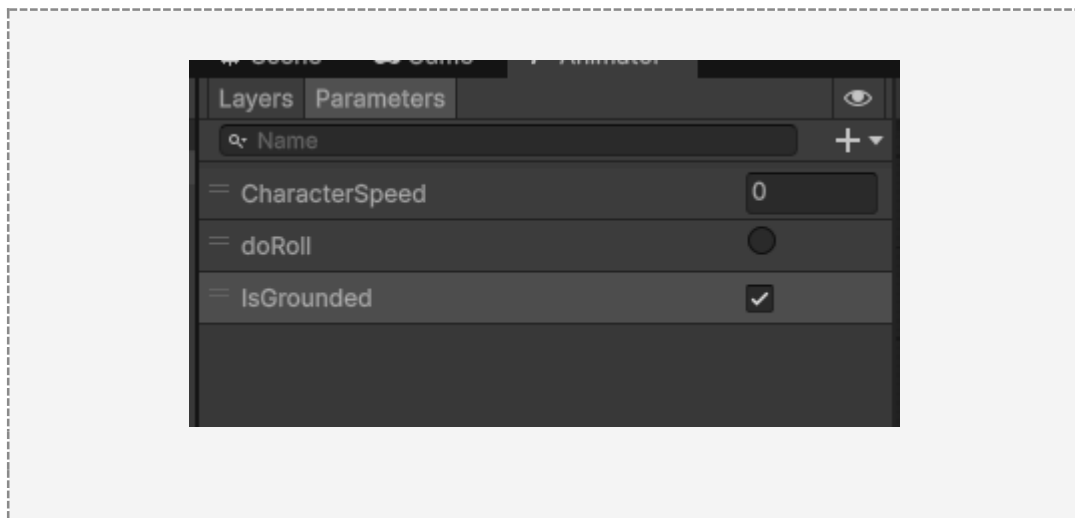
○



- Select the transition from **Falling Idle** back to **Idle/Walking/Running** and set the condition to **IsGrounded== true** and we also want to make sure HasExitTime is checked off.



6. Make sure the parameter IsGrounded is set to true by default



Task 6: Connect Movement to Animation with a PlayerAnimatorController Script

Now we connect the two halves of the lab. The Animator parameters you created (**CharacterSpeed**, **IsGrounded** and **doRoll**) need to be updated every frame from the character's actual movement.

1. Expose the grounded state from PlayerMovement:

- The animation script needs to know whether the player is on the ground, but **isGrounded** is **private** inside PlayerMovement, so other scripts can't see it.
- Open **PlayerMovement** and add this read-only property just below the private variables:

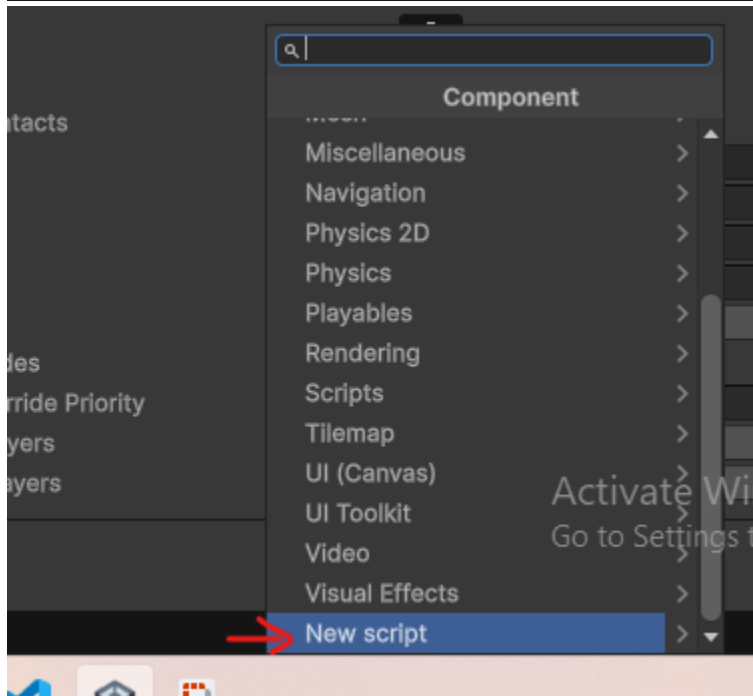
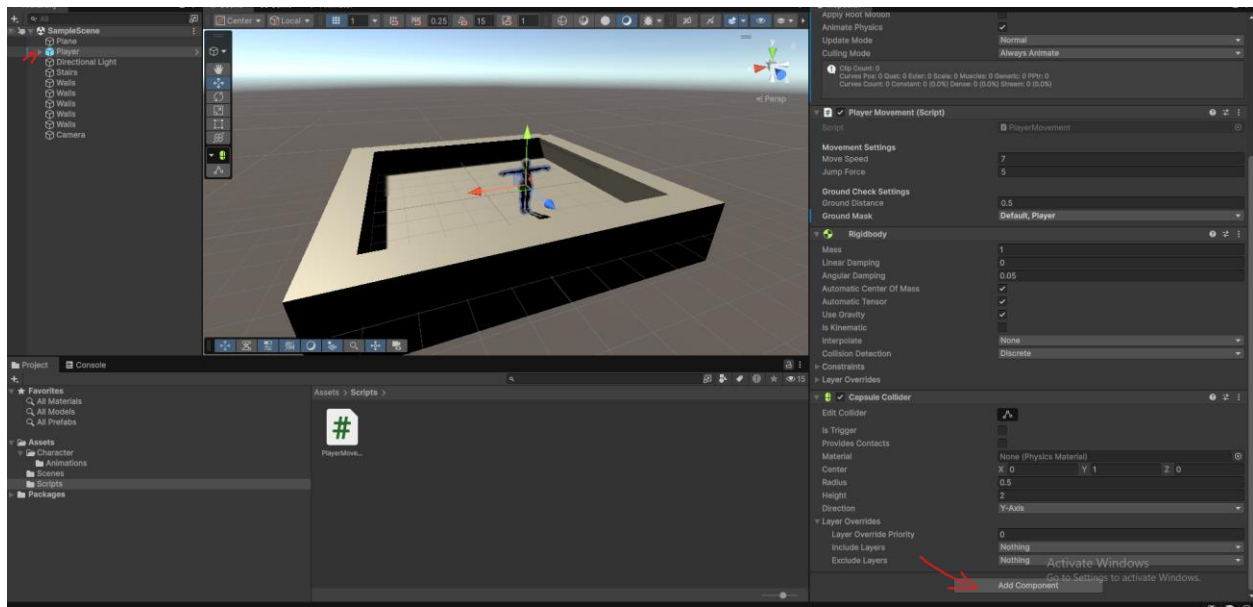
```
// Lets other scripts READ isGrounded, but not change it.  
// "=> isGrounded" means "when asked, return isGrounded".  
public bool IsGrounded => isGrounded;
```

Why a property instead of making the field public? => **isGrounded** means other scripts can read the value but can't overwrite it. Only PlayerMovement is allowed to decide whether the player is grounded.

```
[RequireComponent(typeof(Rigidbody))]  
public class PlayerMovement : MonoBehaviour  
{  
    [Header("Movement Settings")]  
    [SerializeField] private float moveSpeed = 7f;        // units per second  
    [SerializeField] private float turnSpeed = 150f;      // degrees per second  
    [SerializeField] private float jumpForce = 5f;        // strength of the jump  
  
    [Header("Ground Check Settings")]  
    [SerializeField] private float groundDistance = 0.5f; // ray length  
    [SerializeField] private LayerMask groundMask;        // ground layers  
  
    private Rigidbody rb;                                // the physics body we move  
    private Vector3 moveDirection;                       // worked out from input in Update()  
    private float turnInput;                             // horizontal input for rotation  
    private bool isGrounded;                             // true when standing on the ground  
    private bool jumpRequested = false;                  // set in Update(), used later  
  
    public bool IsGrounded => isGrounded;
```

2. Create the PlayerAnimatorController script:

Select the Player, click **Add Component** and create a new script called **PlayerAnimatorController** (the file name must match the class name exactly). Replace its contents with this code:



```
using UnityEngine;

// Sends the player's movement to the Animator every frame
// so the right animation plays at the right time.
public class PlayerAnimatorController : MonoBehaviour
{
    private Animator animator; // plays the animations
    private PlayerMovement movement; // our movement script (for IsGrounded)
    private Rigidbody rb; // used to read how fast the player moves

    private void Start()
    {
```

```

    // Find the components on this same GameObject
    animator = GetComponent<Animator>();
    movement = GetComponent<PlayerMovement>();
    rb = GetComponent<Rigidbody>();
}

private void Update()
{
    // Send the current speed to the blend tree (Idle/Walking/Running).
    // Unity 6 calls the Rigidbody's velocity linearVelocity.
    animator.SetFloat("CharacterSpeed", rb.linearVelocity.magnitude);

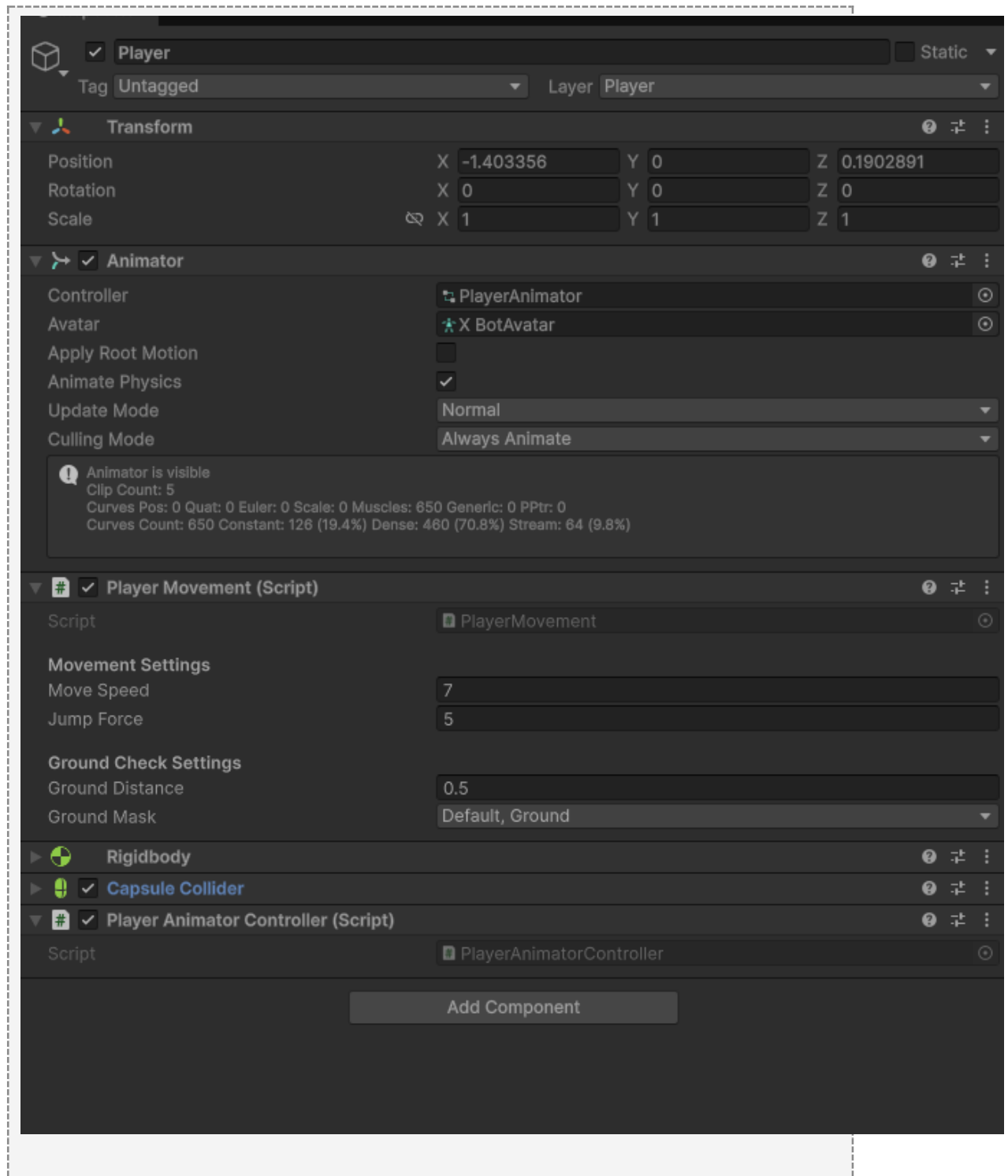
    // Tell the Animator if we're on the ground (controls Falling)
    animator.SetBool("IsGrounded", movement.IsGrounded);

    // When Fire1 (left mouse button / left Ctrl) is released, roll
    if (Input.GetButtonUp("Fire1"))
    {
        animator.SetTrigger("doRoll");
    }
}
}

```

How it works:

- `GetComponent<PlayerMovement>()` finds the script you wrote in Task 2 on the same GameObject.
- `SetFloat("CharacterSpeed", ...)` sends the Rigidbody's speed to the blend tree every frame, so Idle, Walking and Running blend automatically as the character speeds up and slows down.
- `SetBool("IsGrounded", movement.IsGrounded)` passes the raycast result to the Animator, which triggers the Falling transitions from Task 5.
- `SetTrigger("doRoll")` fires the roll when you release **Fire1** (left mouse button or left Ctrl).
- The parameter names in the strings must match the Animator parameters **exactly**, including capital letters.



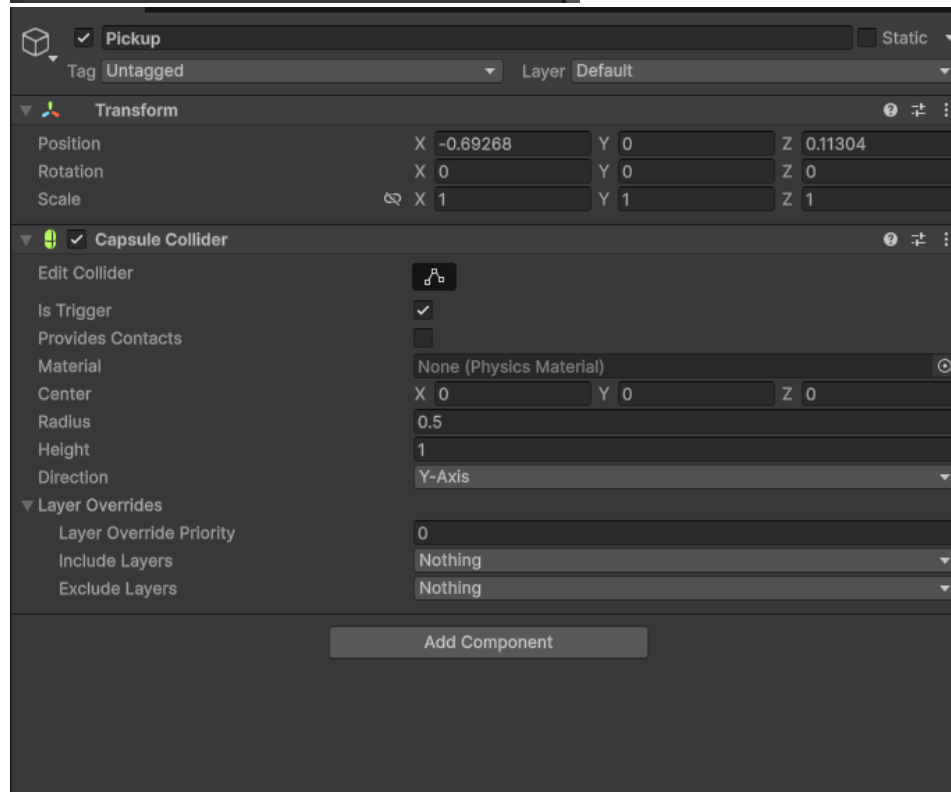
Your Inspector should now show a **Player Movement** component (from Task 2), a **Player Animator Controller** component, a **Rigidbody** and a **Capsule Collider**, with **Apply Root Motion** unchecked.

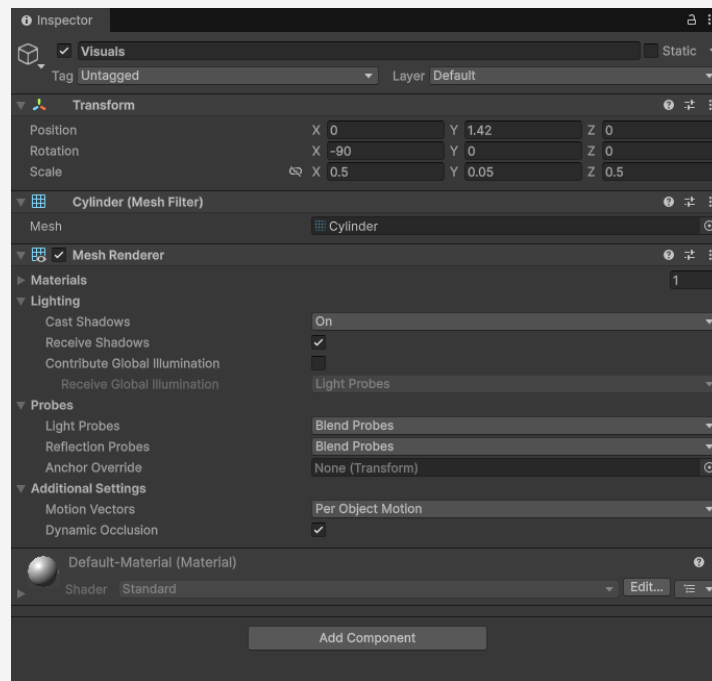
Task 7: Animate a Score Pickup with an Animation Clip

So far every animation has come from imported clips. In this task you will make your **own** animation clip in Unity's Animation window: a coin that spins and bobs up and down. When the player runs over it, it adds points to the player's score and disappears.

1. Build the coin from a parent and a child:

- In the **Hierarchy**, right-click and choose **Create Empty**. Rename it **ScorePickup** and set its **Position Y** to 0.5, somewhere the player can reach.
- Right-click **ScorePickup** and choose **3D Object > Cylinder**, so the cylinder is a **child** of ScorePickup. Rename it **Visuals**.
- On **Visuals**, set **Position** to 0, 0, 0, **Rotation** to 90, 0, 0 and **Scale** to 0.5, 0.05, 0.5 so it looks like a coin standing on its edge.
- On **Visuals**, remove the **Capsule Collider** (right-click the component header > **Remove Component**). The parent will have the collider.
- Optional: create a gold-coloured material and drag it onto **Visuals**.

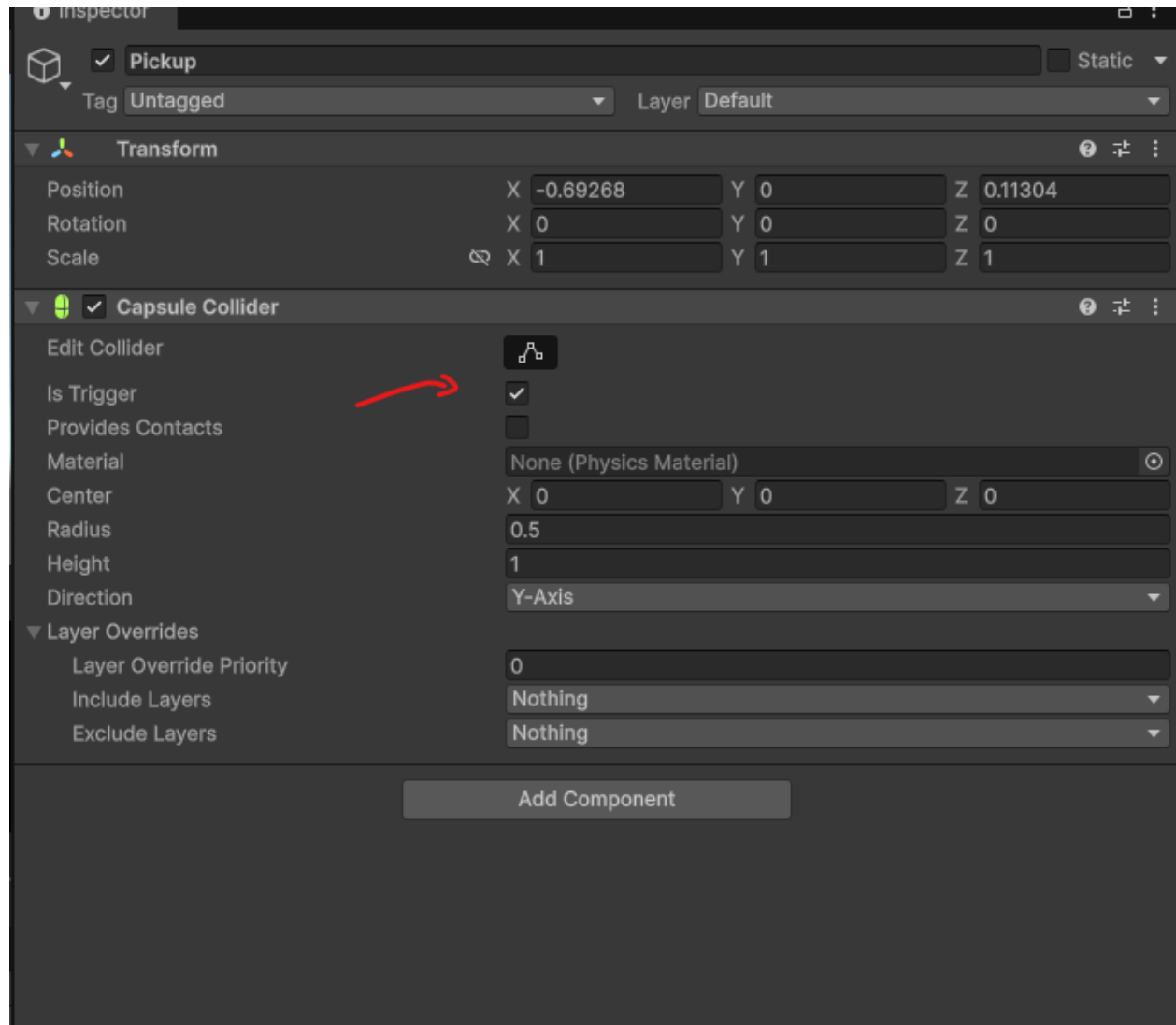




Why a parent and a child? An animation clip stores exact position values. If we animated the object we place in the scene, every copy would snap to the same spot the moment the clip played. Instead we animate the **child's local position**, which is relative to its parent. The parent can then sit anywhere in the level and the coin bobs around that point.

2. Make the coin a trigger:

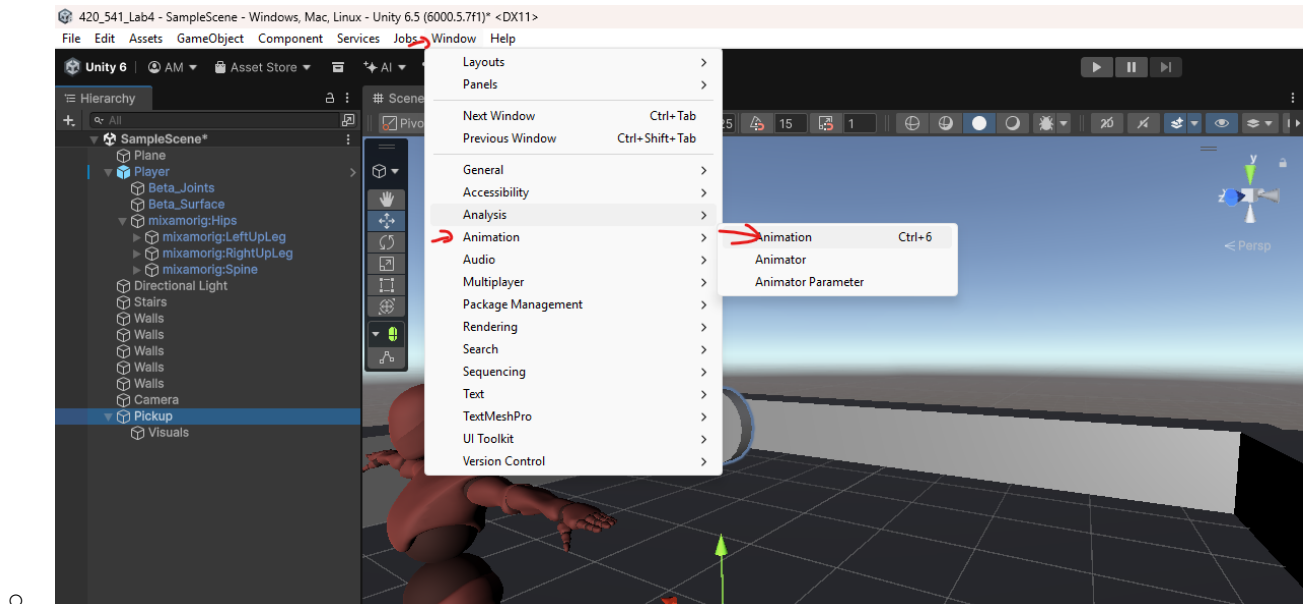
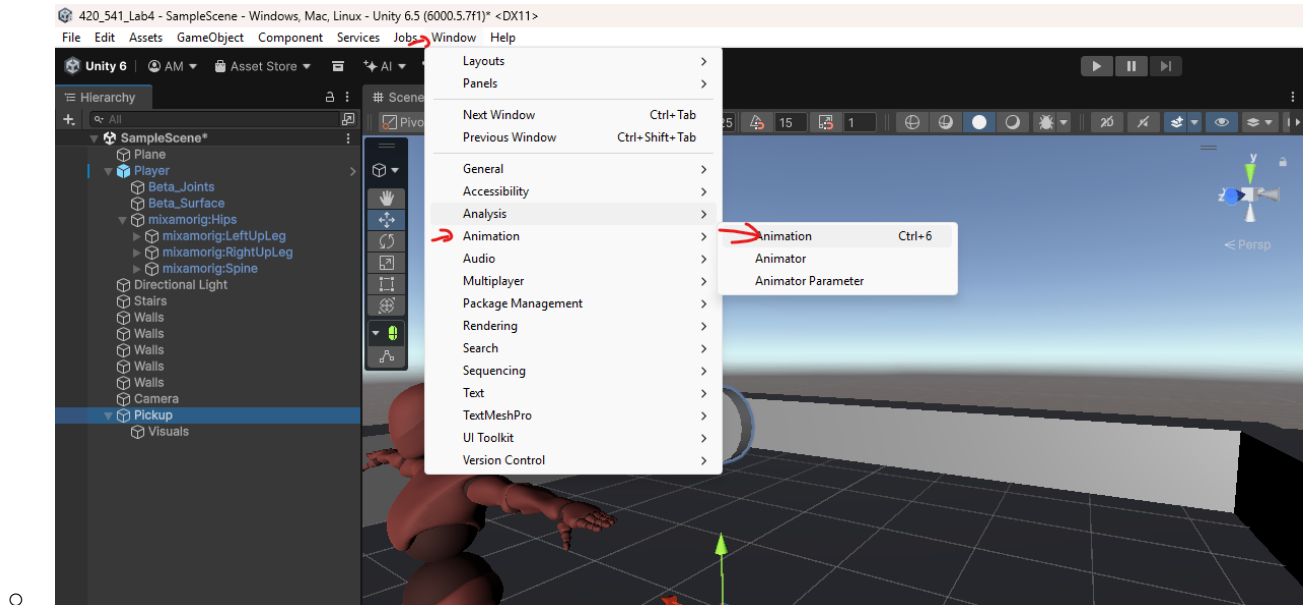
- Select **ScorePickup** (the parent) and click **Add Component > Sphere Collider**.
- Check **Is Trigger** and set **Radius** to 0.5.



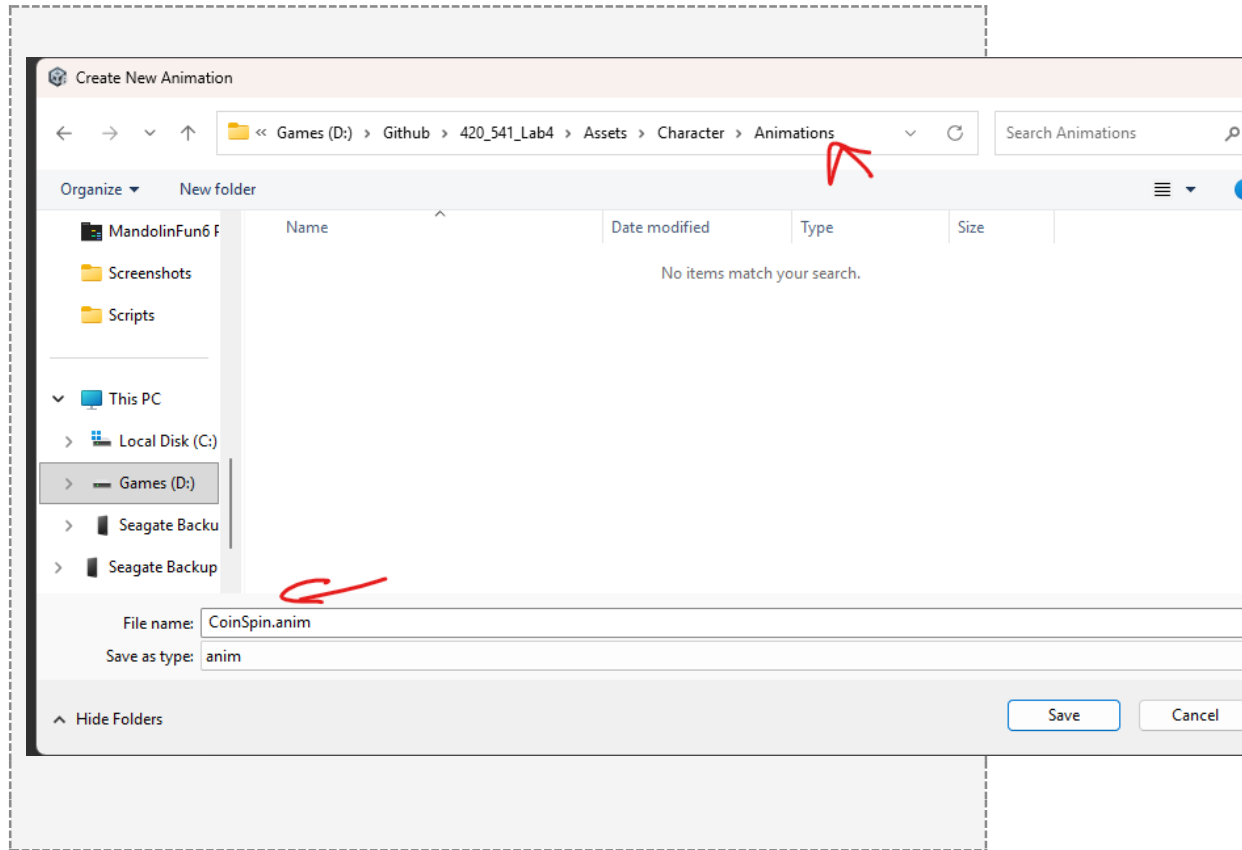
Trigger vs. collider: A normal collider is solid, so the player would bump into the coin and stop. A **trigger** is not solid: things pass through it, but Unity still tells our script the moment something enters it by calling `OnTriggerEnter`. Triggers only fire if one of the two objects has a Rigidbody. Our player already does.

3. Create the animation clip:

- Select **ScorePickup** (the parent) and open **Window > Animation > Animation** (not the Animator window).

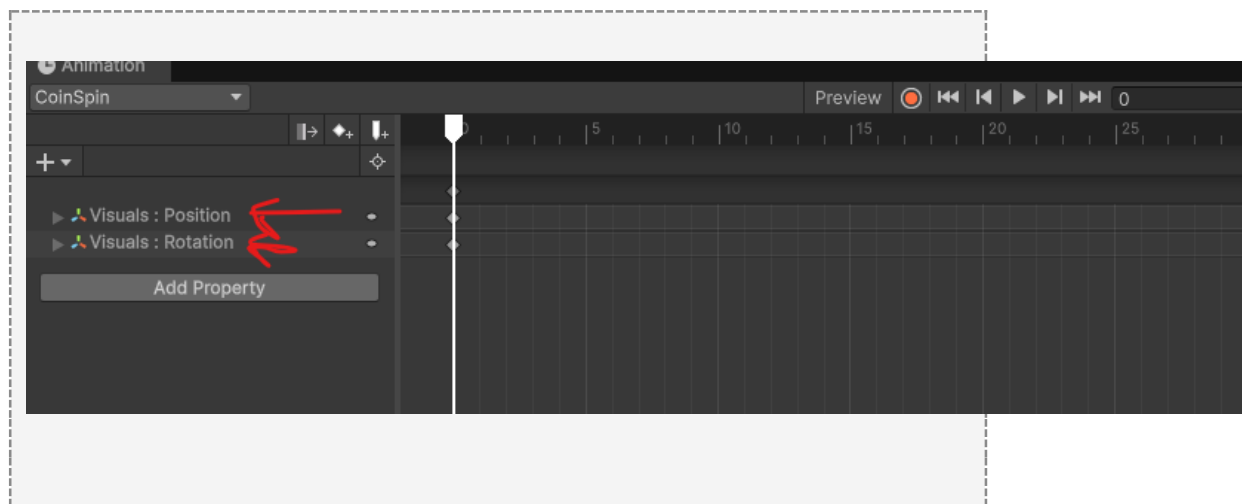


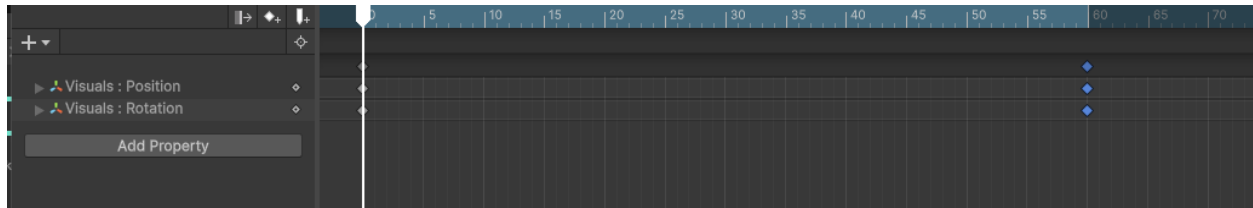
- Click **Create**. Save the clip as **CoinSpin** in your Animation folder.
- Unity automatically adds an **Animator** component to ScorePickup and creates an Animator Controller with **CoinSpin** as its default state. You don't need to set up any states or transitions yourself.



4. Choose which properties to animate:

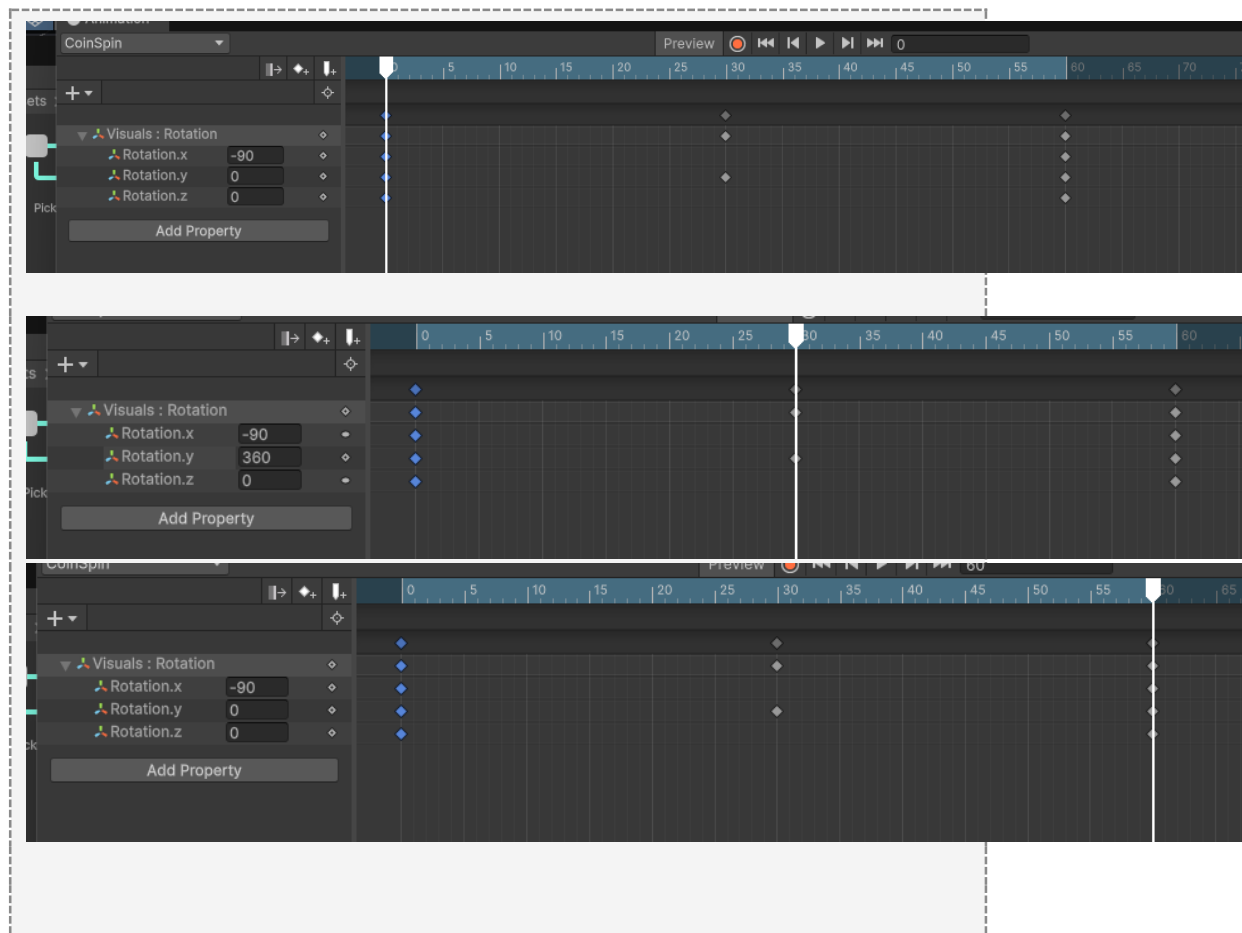
- In the Animation window, click **Add Property**, expand **Visuals > Transform** and click **+** next to **Position**. Do the same for **Rotation**.
- Unity adds keyframes (the diamonds) at the start (0:00) and end (1:00) of the timeline, using the current values.

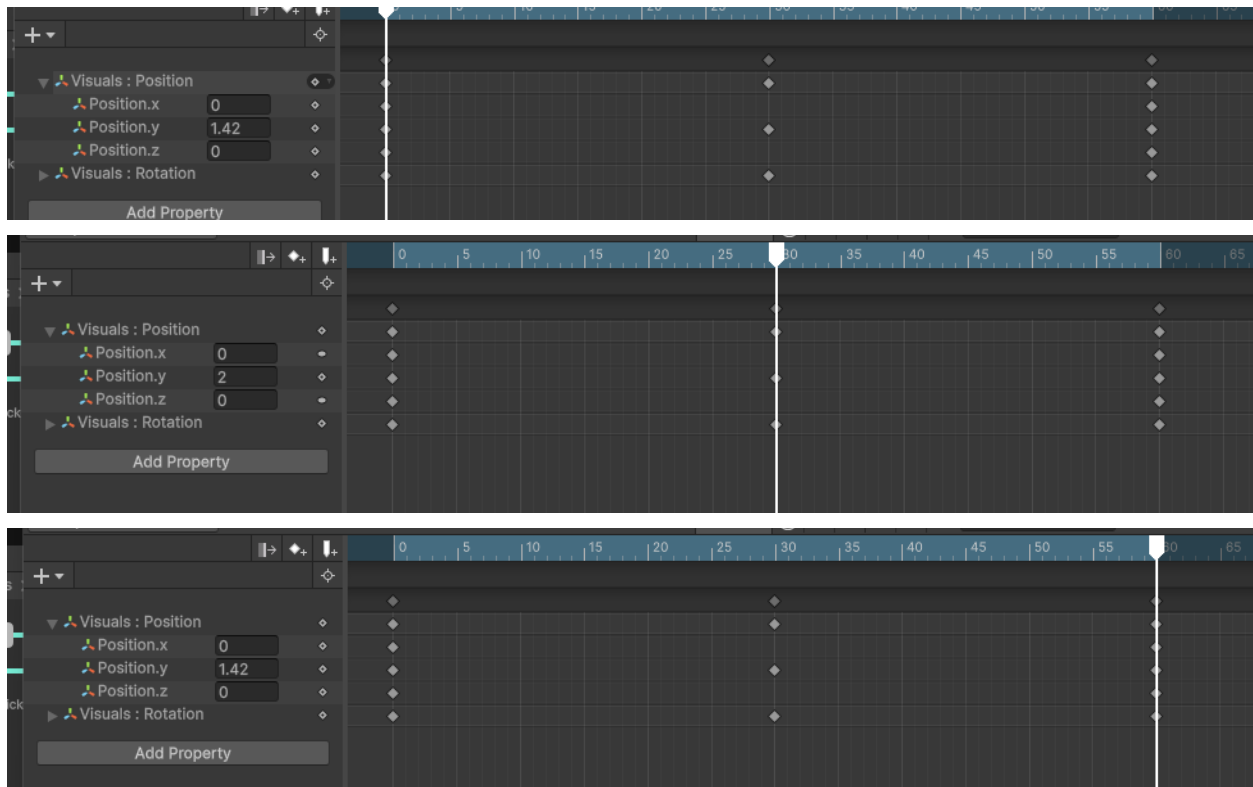




5. Record the spin and the bob:

- Click the red **Record** button. While it's on, any change you make to a property becomes a keyframe.
- **Bob up:** click the timeline at **0:30** to move the playhead there, select **Visuals** in the Hierarchy and set its **Position Y** to 0.25.
- **Spin:** move the playhead to **1:00** and set **Visuals Rotation Y** to 360.
- Click **Record** again to stop recording, then press the **Play** button in the Animation window to preview. The coin should rise, fall back and make one full turn every second.

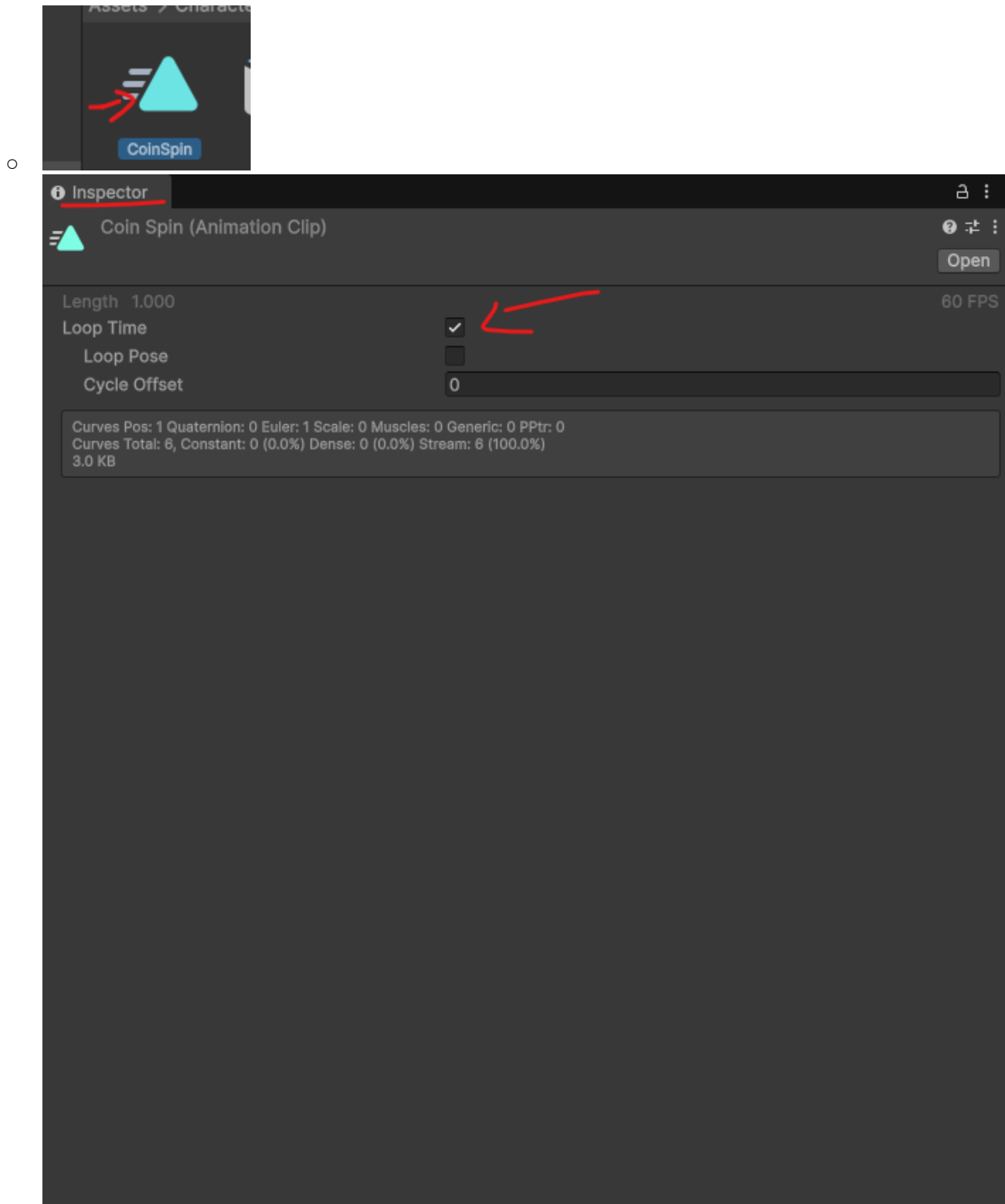




How keyframes work: You only set the important moments (the **keyframes**). Unity fills in every frame in between by smoothly blending the values. This blending is called **interpolation**. Three keyframes are enough for a full second of animation.

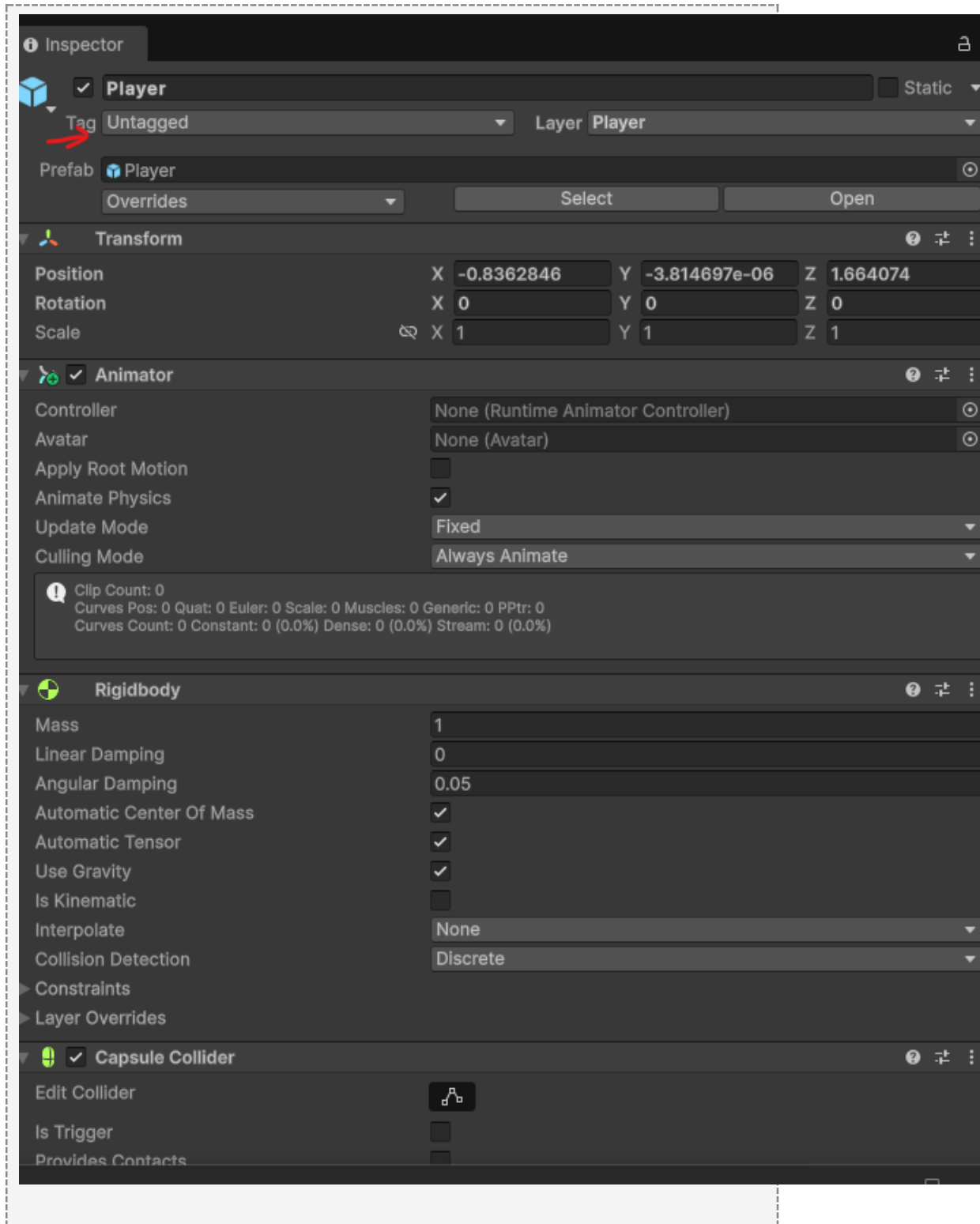
6. Make the clip loop:

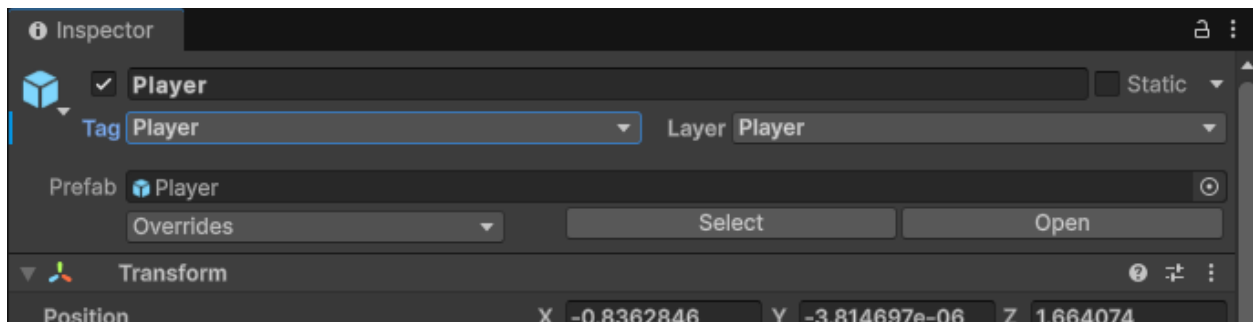
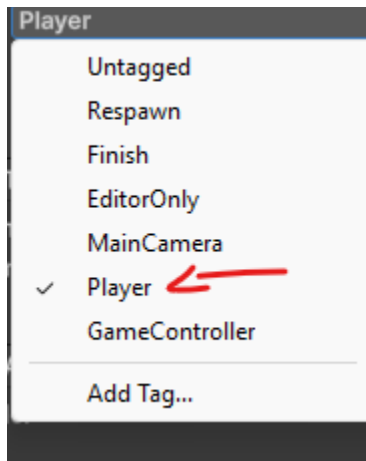
- Select the **CoinSpin** clip in the Project window and make sure **Loop Time** is checked in the Inspector, so the coin keeps spinning forever.



7. Create the ScoreManager script:

- Select the **Player**. At the top of the Inspector, set its **Tag** to **Player** (a built-in tag). The coins use this tag to recognise the player.





- Create an empty game object, click **Add Component**, create a new script called **ScoreManager** and replace its contents with:

```
using UnityEngine;

// Keeps track of the player's score.
// It's a singleton: there is only ever one, reachable from any script
// through ScoreManager.Instance.
public class ScoreManager : MonoBehaviour
{
    // The single shared instance that any script can reach.
    // "static" means it belongs to the class, not to one object.
    public static ScoreManager Instance { get; private set; }

    // Anyone can read the score, only ScoreManager can change it
    public int Score { get; private set; }

    // Awake runs before any Start(), so Instance is ready before coins need it
    private void Awake()
    {
        // If a ScoreManager already exists, remove this extra component
        // (only the script, not the whole Player)
        if (Instance != null && Instance != this)
        {
            Destroy(this);
            return;
        }

        // Otherwise, this is the one and only ScoreManager
    }
}
```

```

        Instance = this;
    }

    // Called by coins when they're collected
    public void AddScore(int amount)
    {
        Score += amount;

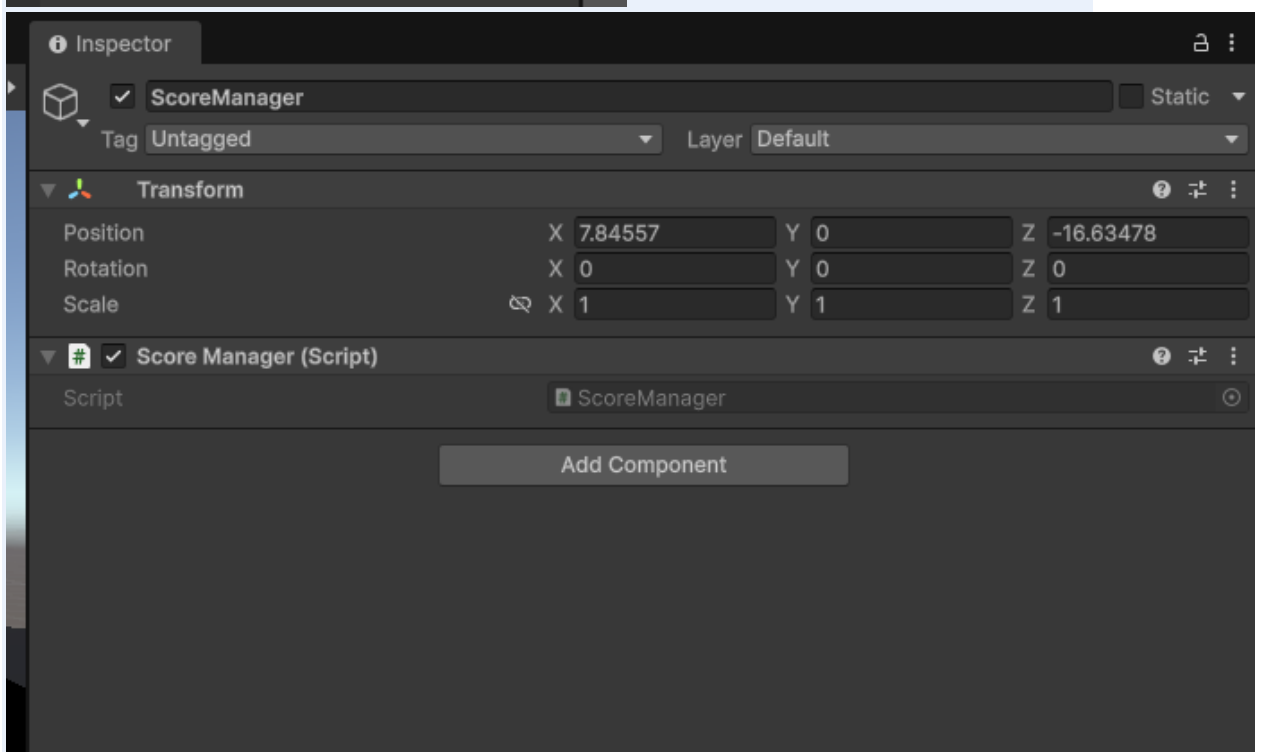
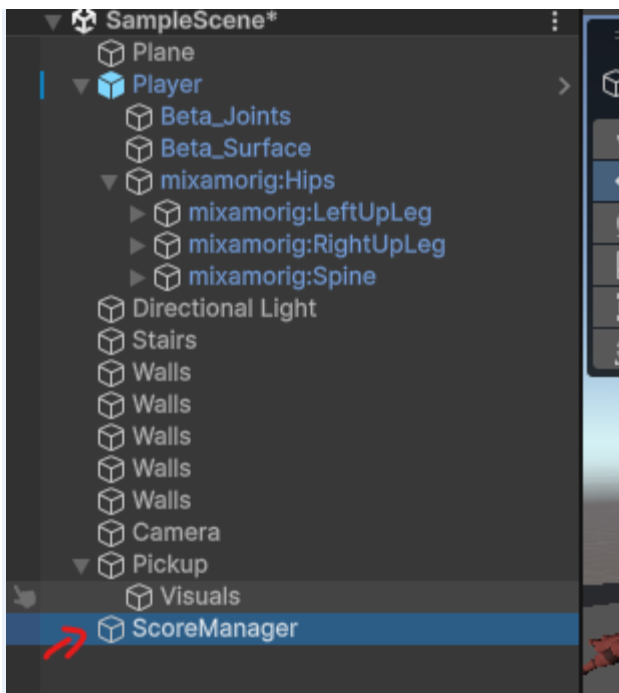
        // Print the new total to the Console window
        Debug.Log("Score: " + Score);
    }
}

```

How it works:

- **Instance** is a **static** property, which means it belongs to the **ScoreManager** class itself rather than to one object. Any script can write **ScoreManager.Instance** to reach the manager. **private set** means only this class can assign it.
- **Awake()** runs before any **Start()** method. It stores this manager in **Instance**. If one already exists, it destroys this extra **component** with **Destroy(this)**, not the whole **Player**, so there is only ever one manager.
- **public int Score { get; private set; }** lets any script read the score (for a UI later) but only **ScoreManager** can change it.
- **AddScore()** adds the points and prints the new total to the **Console** window.

What is a singleton? A **singleton** is a class that only ever has one instance, which everyone can reach through a single global access point (**Instance**). It's useful for managers that there should only be one of, like a game, audio or score manager. The trade-off is that any script can call it from anywhere, so use singletons for true one-of-a-kind managers, not for everything.



8. Create the ScorePickup script:

- Select **ScorePickup** (the parent), click **Add Component**, create a new script called **ScorePickup** and replace its contents with:

```
using UnityEngine;

// A coin that adds points to the score when the player touches it.
// Needs a collider with Is Trigger ticked on the same GameObject.
public class ScorePickup : MonoBehaviour
{
    // How many points this coin is worth (change it in the Inspector)
    [SerializeField] private int points = 10;

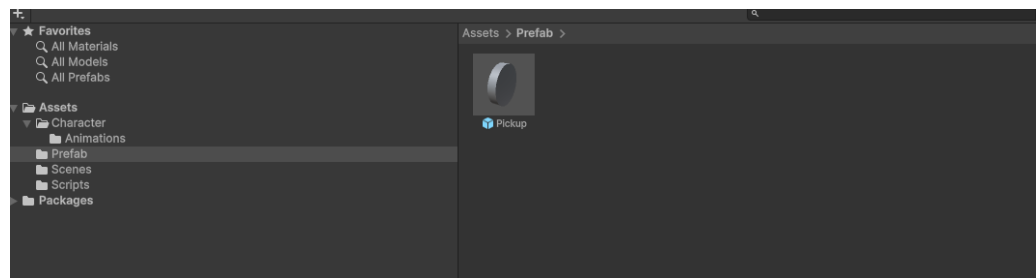
    // Called automatically when something enters this trigger
    private void OnTriggerEnter(Collider other)
    {
        // Only react to the player, ignore everything else
        if (other.CompareTag("Player"))
        {
            // Add our points through the ScoreManager singleton
            ScoreManager.Instance.AddScore(points);

            // Remove the coin from the scene: it's collected once and gone
            Destroy(gameObject);
        }
    }
}
```

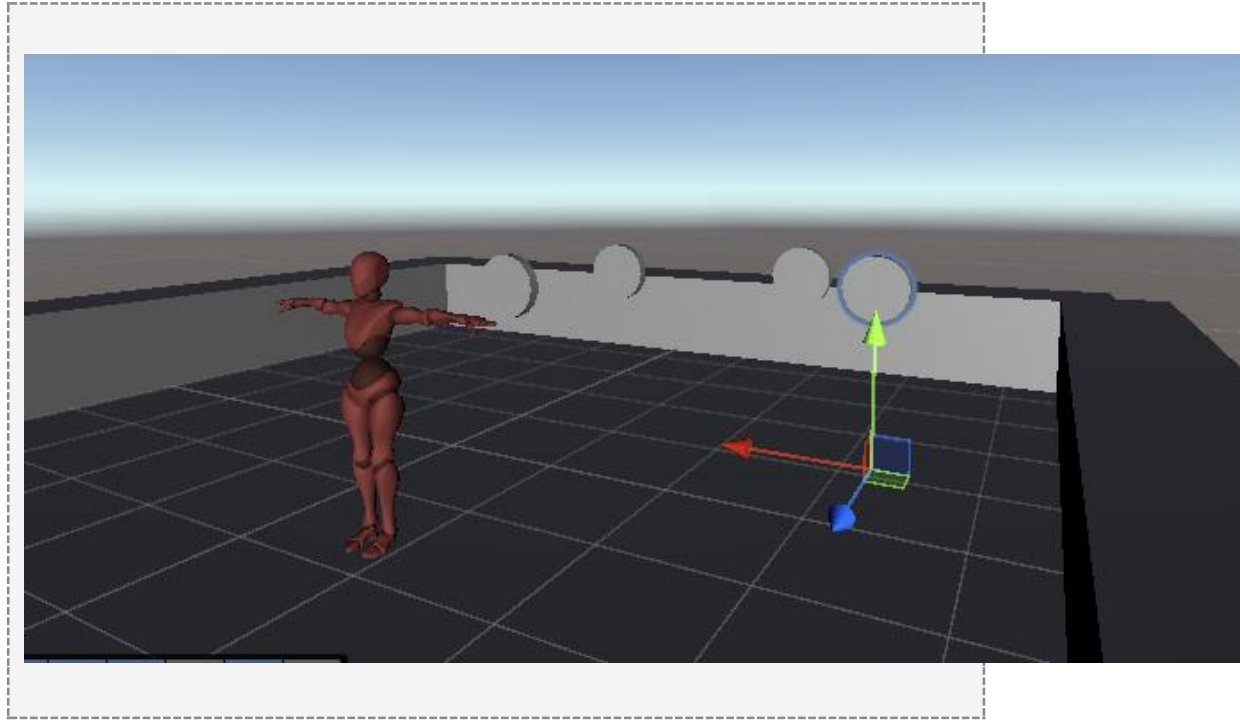
- When the player touches the coin, it adds **points** to the score and then **destroys** itself.
- Thanks to the singleton, the coin doesn't need a reference to the player or a **GetComponent** call to find the ScoreManager.

9. Turn the coin into a prefab and place a few:

- Drag **ScorePickup** into your **Assets** folder to make a prefab, then drag several copies around the level.



- Each copy bobs around its own position because only the child is animated.



10. Test the coins:

- Open the **Console** window (**Window > General > Console**) and press **Play**.
- The coins should spin and bob. Run through one: it should disappear and the Console should show **Score: 10**, then 20, 30...
- **All the coins jump to the same spot?** You animated **ScorePickup** instead of **Visuals**. The clip's properties should read **Visuals: Position** and **Visuals: Rotation**.
- **The coin doesn't animate?** Check that ScorePickup has an **Animator** with the CoinSpin controller, and that **Loop Time** is on.
- **Nothing happens when you touch it?** Check that the Sphere Collider on the parent has **Is Trigger** ticked and the Player's tag is **Player**.

11. Check your understanding:

- What would go wrong if the animation clip changed the position of ScorePickup itself instead of its Visuals child?

All the coins would start bobbing at the position where the coin was animated.

Final Steps:

1. Test Your Project:

- Press **Play** in Unity. Move with WASD / arrow keys and check that the character blends from Idle to Running, jumps with **Space**, plays Falling while in the air and lands back into Idle/Walking/Running, and rolls when you click **Fire1**.
- Run through a coin and check that it spins and bobs, disappears when collected, and the Console shows the score going up.
- Select the Player while the game is running and open the **Animator** window to watch the parameters and active state change live. This is the fastest way to debug a transition that doesn't fire.

2. Submission:

- Save your fork of the repo.
Submit your Assets folder zipped with this filled in and saved document in the assets folder.

https://github.com/J0YSICK/420_541_Lab5_2026.git